



University
of Glasgow

A WORLD
TOP 100
UNIVERSITY

Introduction to Data Science and Systems

Probabilities III – Continuous Random Variables

Dr Nicolas Pugeault

WORLD
CHANGING
GLASGOW

THE SUNDAY TIMES
GOOD
UNIVERSITY
GUIDE
2024
SCOTTISH
UNIVERSITY
OF THE YEAR



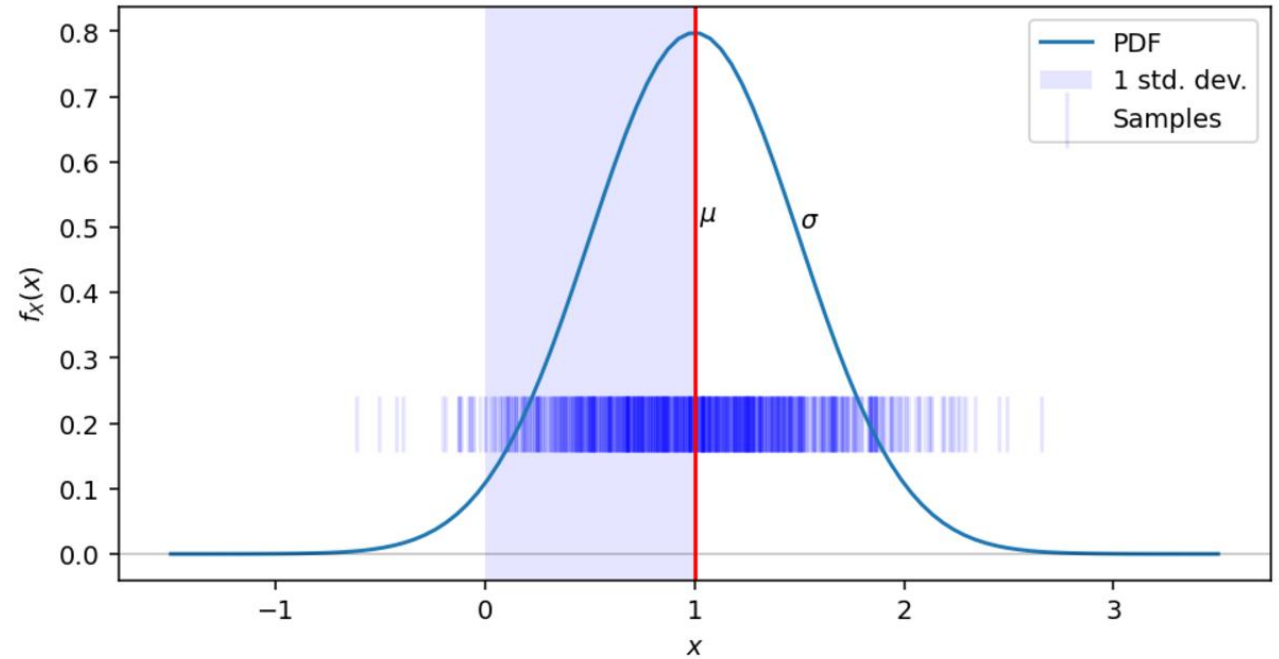
Today's learning outcomes

- the specific problems of continuous random variables as compared to discrete random variables
- how to model simple data using normal distributions and the central limit theorem
- what multivariate distributions are
- what Monte Carlo approaches are, and how expectation can be approximated
- what population, parameters, statistics and samples are
- how estimators and maximum likelihood estimation work
- MCMC approaches to sampling posteriors in Bayesian inference



University
of Glasgow

Continuous random variables



Problems with continuous variables

Continuous random variables are defined by a **PDF (probability density function)**, rather than a PMF (probability *mass* function). A PMF is essentially just a vector of values, but a PDF is **a function mapping any interval in its domain to a probability**. This brings a number of complexities:

- The probability of any specific value is $P(X=x)=0$: zero for every possible x , yet any value in the *support* of the distribution function (everywhere the PDF is non-zero) is possible.
- There is no direct way to sample from the PDF in the same way as we did for the PMF. But there are several tricks for sampling from continuous distributions.
- We cannot estimate the true PDF from simple counts of observations like with the empirical distribution.
- How do we do computations with continuous PDFs using Bayes' Rule?
- Simple discrete distributions don't have a concept of dimension. But we can have continuous values in $\mathbb{R}$, or in vector spaces $\mathbb{R}^n$

Probability distribution functions

The PDF $f_X(x)$ of a random variable X maps a value x (which might be a real number, or a vector, or any other continuous value) to a single number, the density at the point.

It is a function (assuming a distribution over real vectors) $\mathbb{R}^n \rightarrow \mathbb{R}^+$, where $\mathbb{R}^+$ is the positive real numbers, and

$$\int_{\mathbb{R}^n} f_X(x) dx = 1.$$

While a PMF can have outcomes with a probability of at most 1, it is *not* the case that the maximum value of a PDF is $f_X(x) \leq 1$ - *just that the integral of the PDF be 1*.

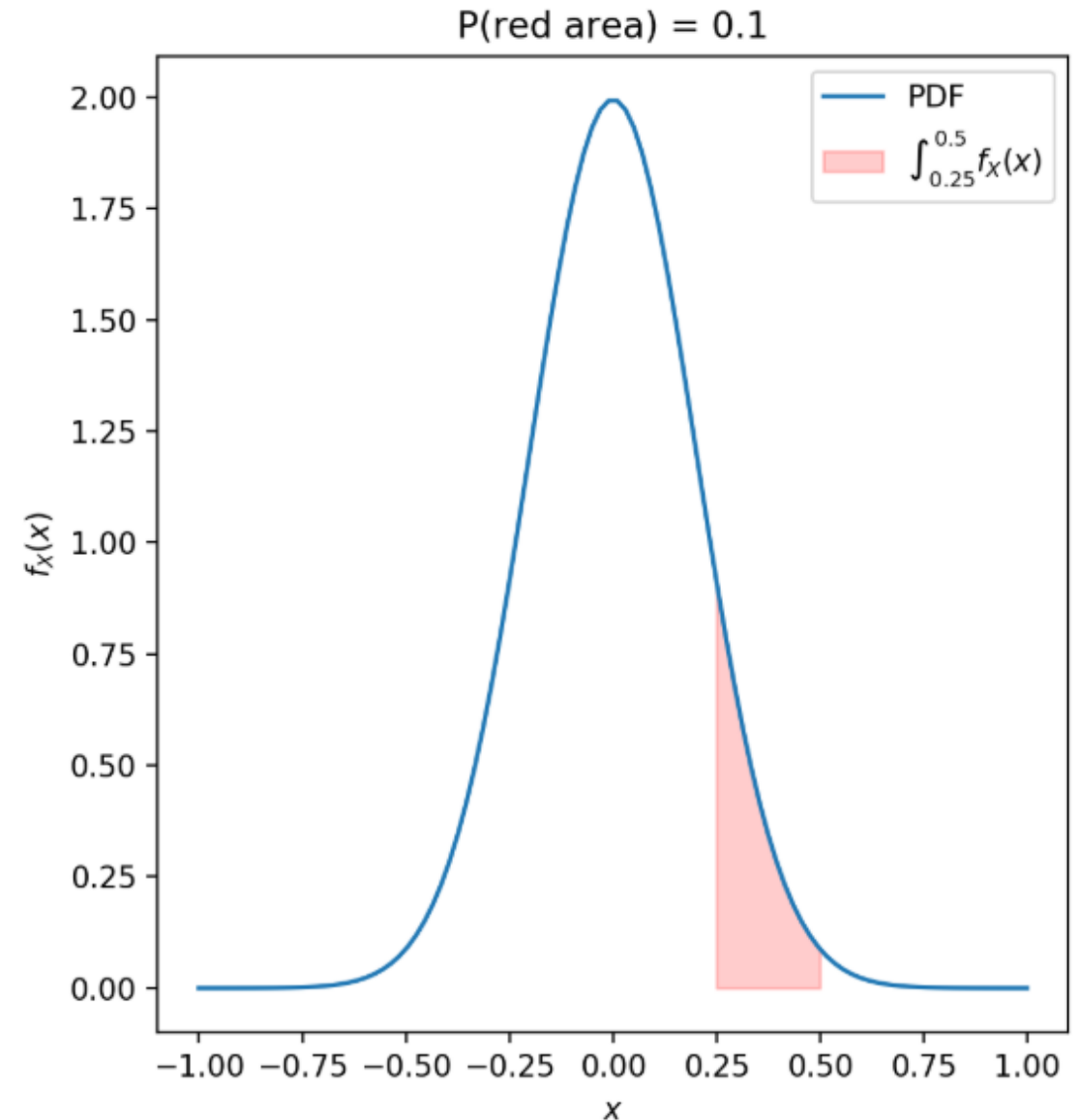


Probability distribution functions

The value of the PDF at any point is **not** a probability, because the probability of a continuous random variable taking on any specific number must be zero.

Instead, we can say that the probability of a continuous random variable X lying in a range (a,b) is:

$$P(X \in (a,b)) = (a < X < b) = \int_a^b f_X(x)$$



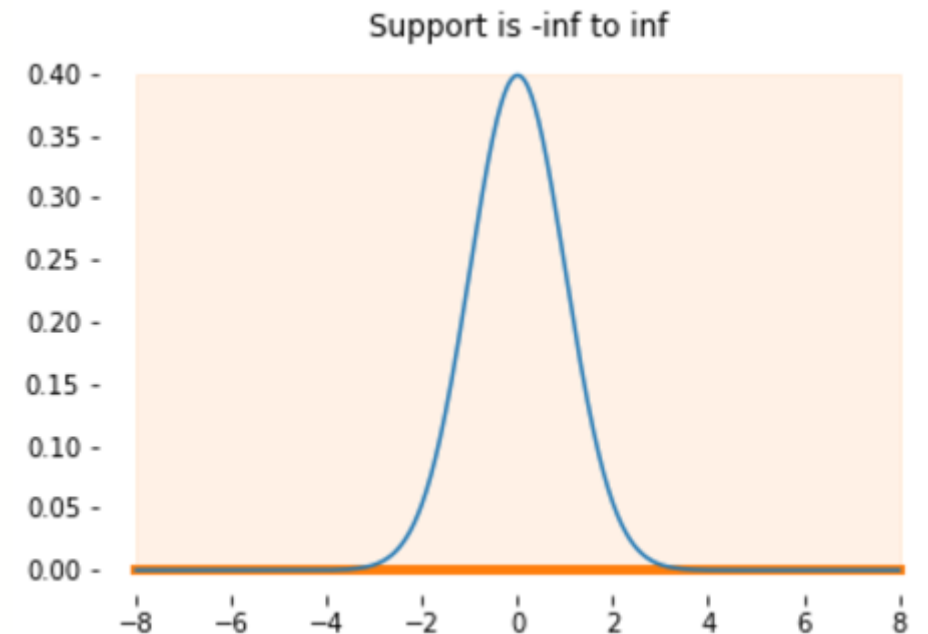
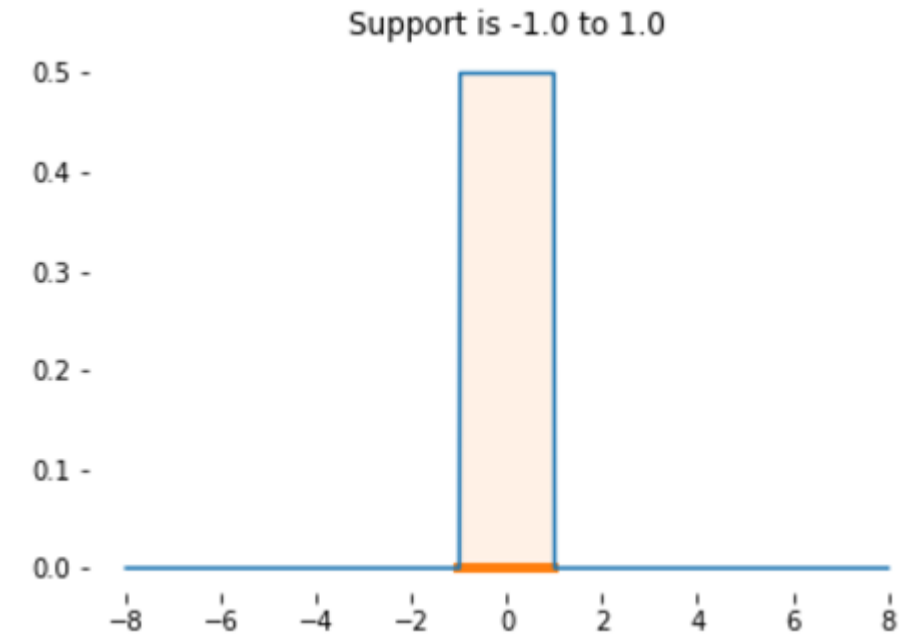


Support

The **support** of a PDF is the domain it maps from where the density is non-zero.

$$\text{supp}(x) = x \text{ such that } f_X(x) > 0$$

- Some PDFs have density over a fixed interval, and have zero density everywhere else (eg, uniform distribution). This is called **compact support**.
- Some PDFs have non-zero density over an infinite domain (eg, normal distribution). This is **infinite support**.





Cumulative distribution function (CDF)

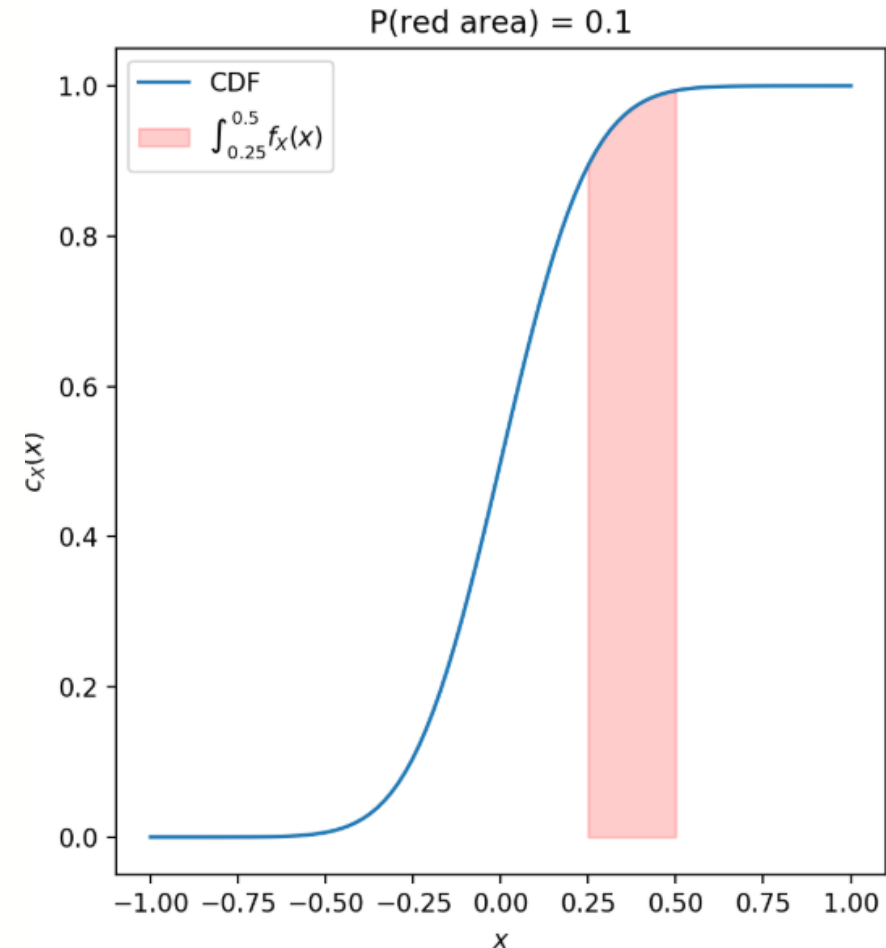
The **cumulative distribution function** or CDF of a real-valued random variable is

$$F_X(x) = \int_{-\infty}^x f_X(x) = P(X \leq x)$$

- the CDF always maps x to $[0,1]$.
- For any given value $F_X(x)$ tells us how much probability mass there is that is less than or equal to x .
- Given a CDF, we can now answer questions, like: what is the probability that random variable X takes on a value between 3.0 and 4.0?

$$P(3 \leq X \leq 4) = F_X(4) - F_X(3)$$

- This *is* a probability.



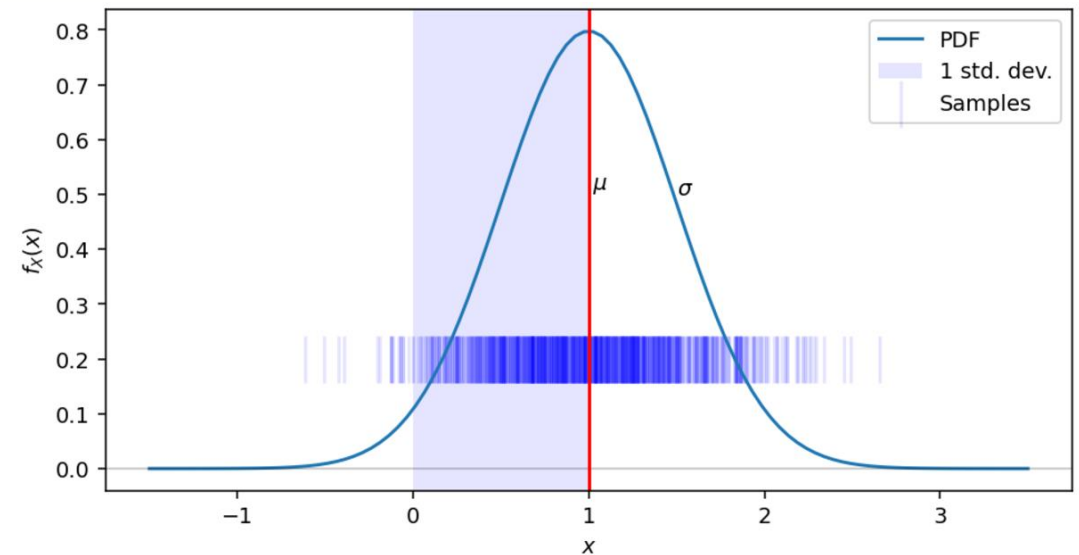


Example: The Normal Distribution

The most ubiquitous of all continuous PDFs is the **normal** or **Gaussian** distribution. It has a density given by the PDF:

$$f_X(x) = \frac{1}{Z} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

You do not need to remember this formula, but it is very useful to know that it is essentially just e^{-x^2} with some scaling factors to normalise it -- this is called the **squared exponential function**.



Side note: $Z = \sqrt{2\pi\sigma^2}$, but you do not need to remember this.



Example: The Normal Distribution (cont'd)

We use a shorthand notation to refer to the distribution of continuous random variables,

variable $\sim$ distribution(parameters),

where $\sim$ is read as "distributed as".

For a normal distribution this is:

$$X \sim \mathcal{N}(\mu, \sigma^2),$$

which is read as:

"Random variable X is distributed as [N]ormal with mean μ and variance σ^2 "

Note: There are various other symbols used for other continuous distributions, including $\Gamma(\alpha, \beta)$, $\beta(\alpha, \beta)$, $t(v)$, $\chi^2(k)$, ..., which we will not cover in this course.

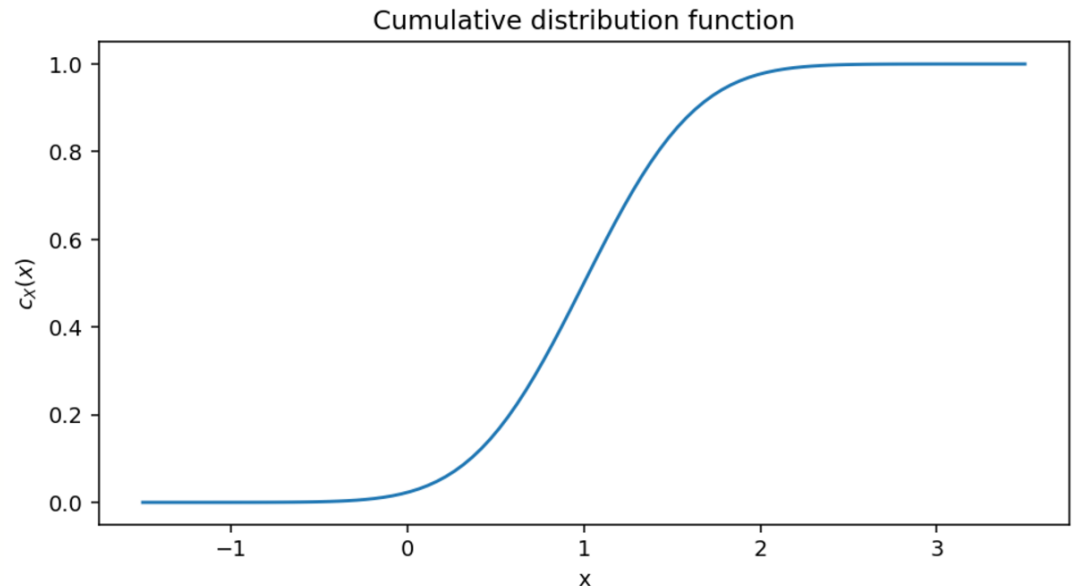
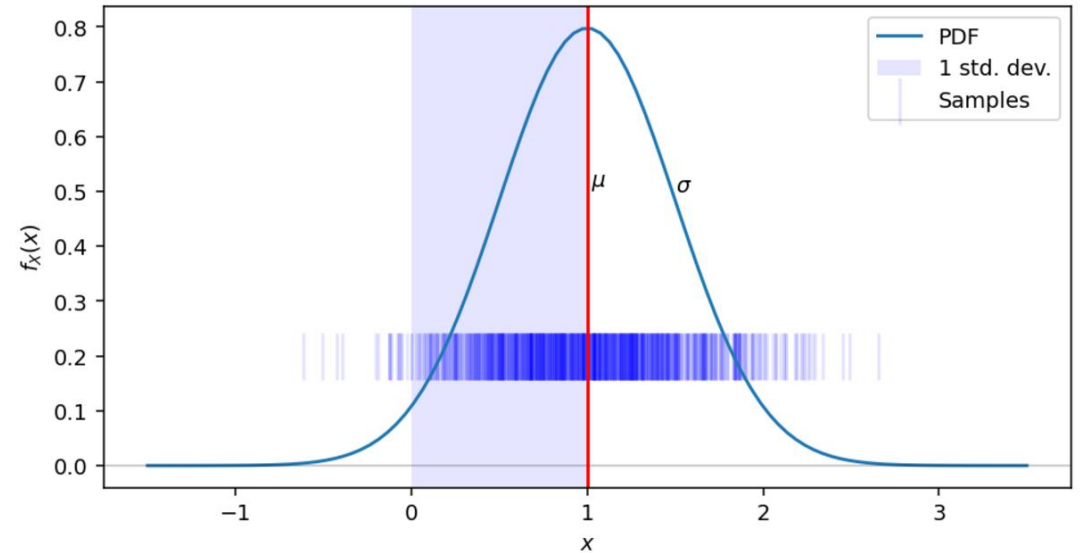


Location and scale

The normal distribution places the point of highest density to its center μ (the "mean"), with a spread defined by σ^2 (the "variance").

This can be thought of the **location** and **scale** of the density function.

Most standard continuous random variable PDFs have a location (where density is concentrated) and scale (how spread out the density is).





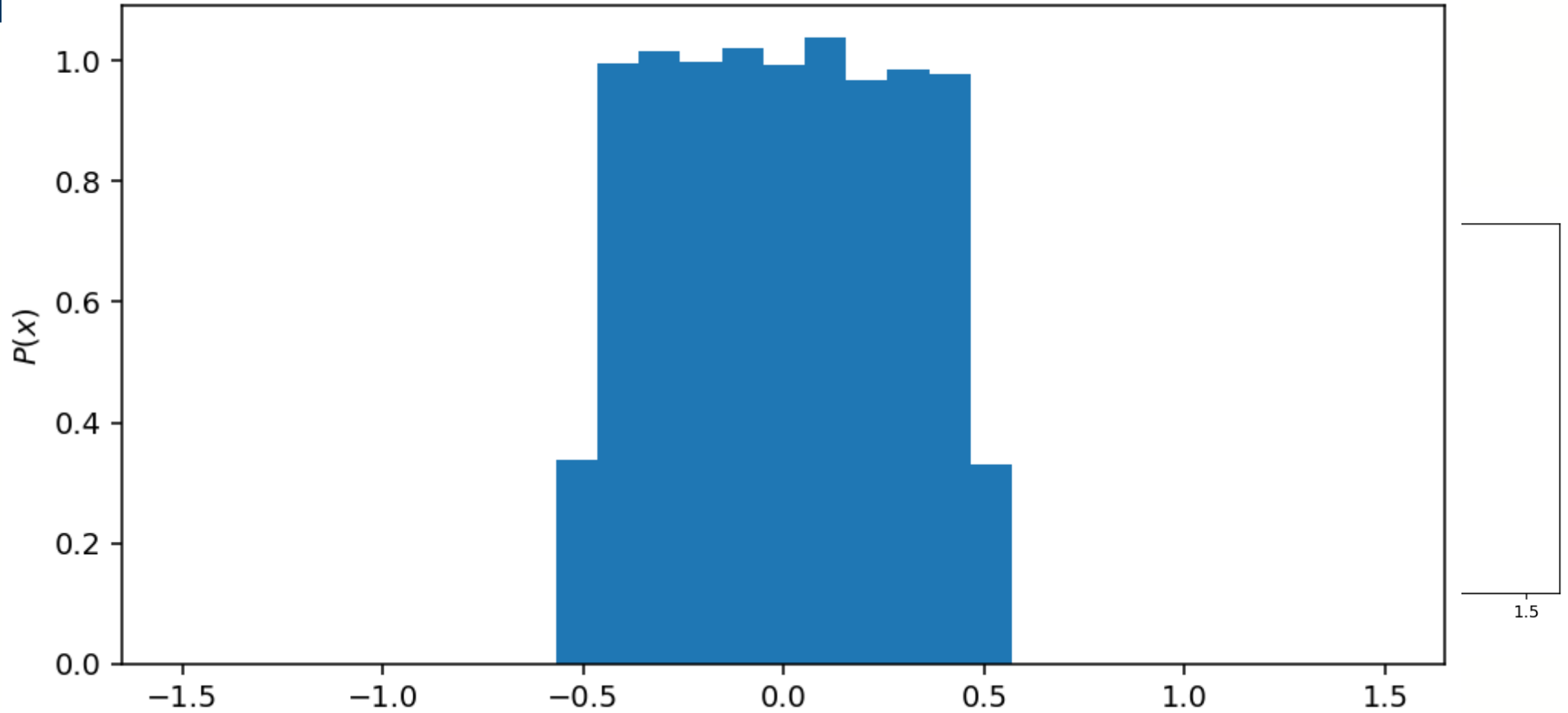
Normal modelling

It seems that this might be a very limiting choice but there are two good reasons for this to work well as a model in many contexts:

1. Normal variables have very nice mathematical properties and are easy to work with analytically (i.e. without relying on numerical computation).
2. The *central limit theorem* tells us that any sum of random variables (however they are distributed) will tend to a *Levy stable distribution* as the number of variables being summed increases. For most random variables encountered, this means the normal distribution (one specific Levy stable distribution).



(normalised) sum of 1 uniform variables

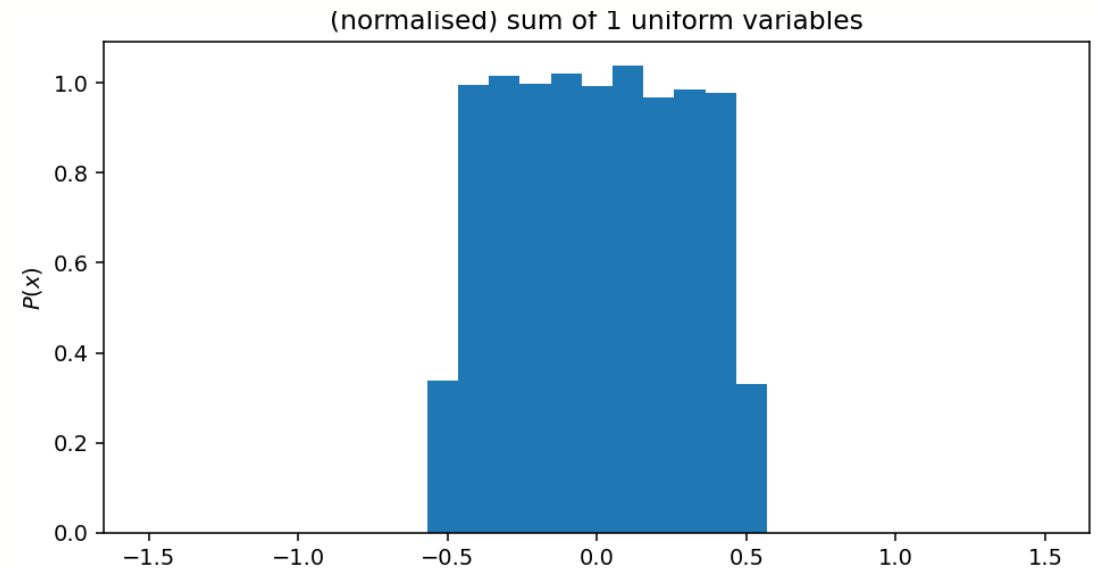




Central limit theorem

If we form a sum of many random variables $Y=X_1+X_2+X_3+\dots$, then for almost any PDF that each of $X_1, X_2, \dots$ might have, the PDF of Y will be approximately normal, $Y \sim N(\mu, \sigma^2)$.

This means that any process that involves a mixture of many random components will tend to be Gaussian under a wide variety of conditions.

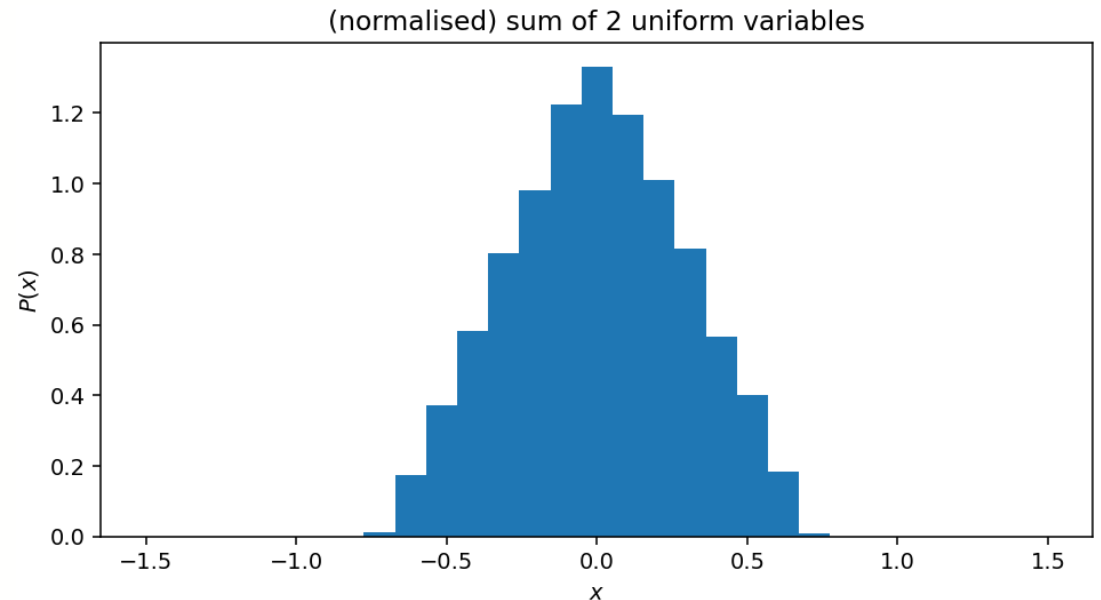




Central limit theorem

If we form a sum of many random variables $Y=X_1+X_2+X_3+\dots$, then for almost any PDF that each of $X_1, X_2, \dots$ might have, the PDF of Y will be approximately normal, $Y \sim N(\mu, \sigma^2)$.

This means that any process that involves a mixture of many random components will tend to be Gaussian under a wide variety of conditions.

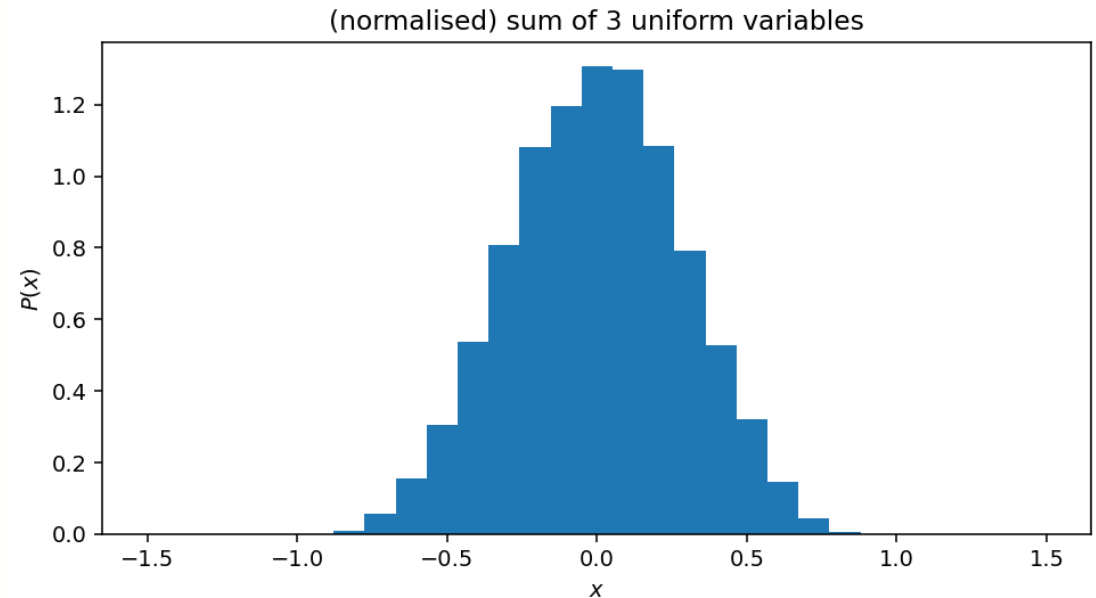




Central limit theorem

If we form a sum of many random variables $Y=X_1+X_2+X_3+\dots$, then for almost any PDF that each of $X_1, X_2, \dots$ might have, the PDF of Y will be approximately normal, $Y \sim N(\mu, \sigma^2)$.

This means that any process that involves a mixture of many random components will tend to be Gaussian under a wide variety of conditions.

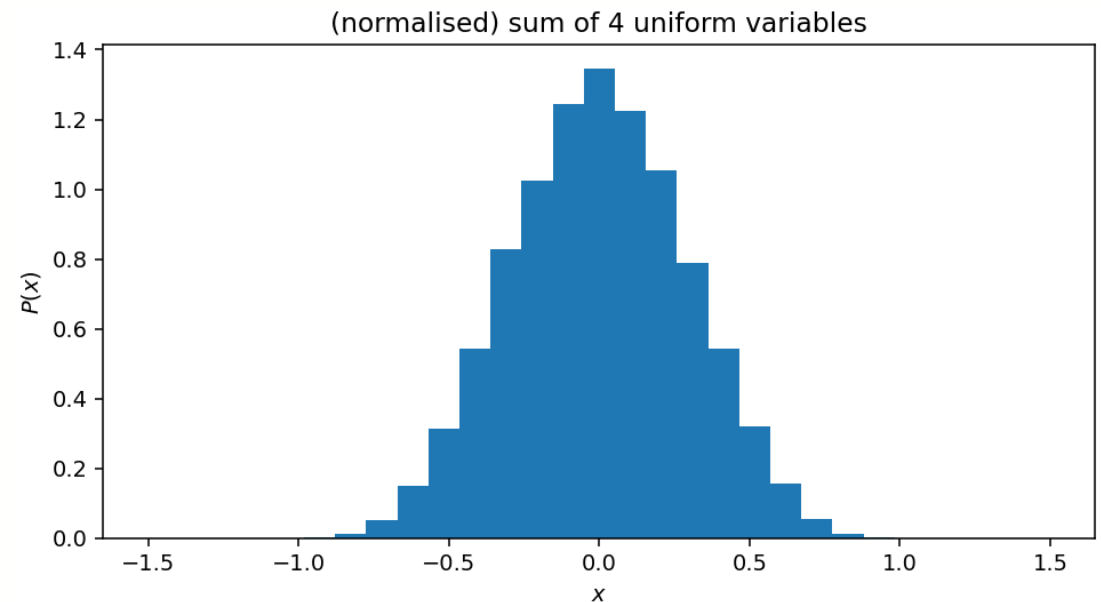




Central limit theorem

If we form a sum of many random variables $Y=X_1+X_2+X_3+\dots$, then for almost any PDF that each of $X_1, X_2, \dots$ might have, the PDF of Y will be approximately normal, $Y \sim N(\mu, \sigma^2)$.

This means that any process that involves a mixture of many random components will tend to be Gaussian under a wide variety of conditions.

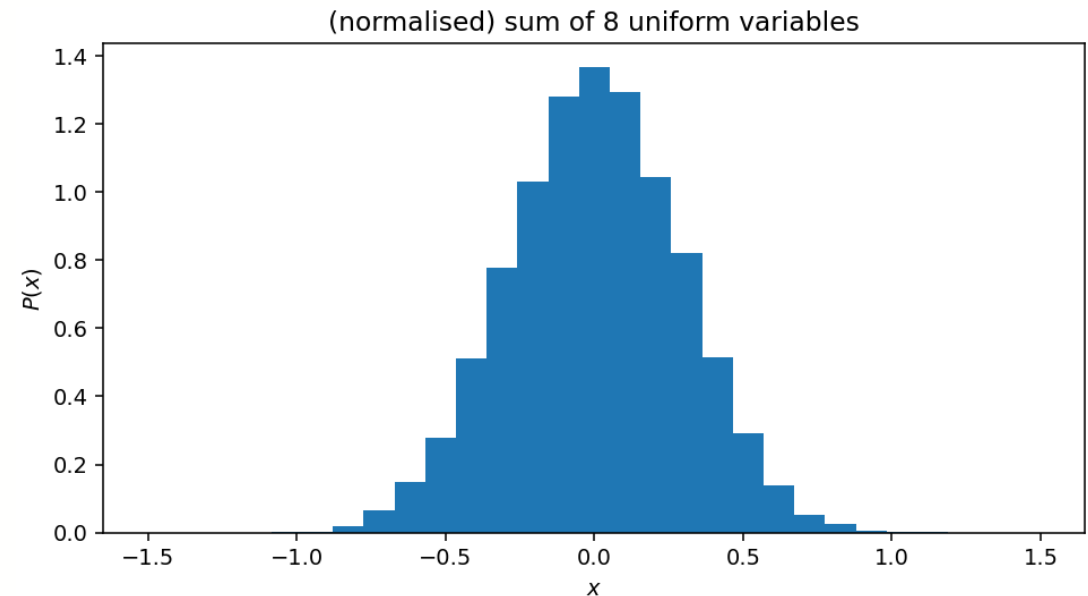




Central limit theorem

If we form a sum of many random variables $Y=X_1+X_2+X_3+\dots$, then for almost any PDF that each of $X_1, X_2, \dots$ might have, the PDF of Y will be approximately normal, $Y \sim N(\mu, \sigma^2)$.

This means that any process that involves a mixture of many random components will tend to be Gaussian under a wide variety of conditions.

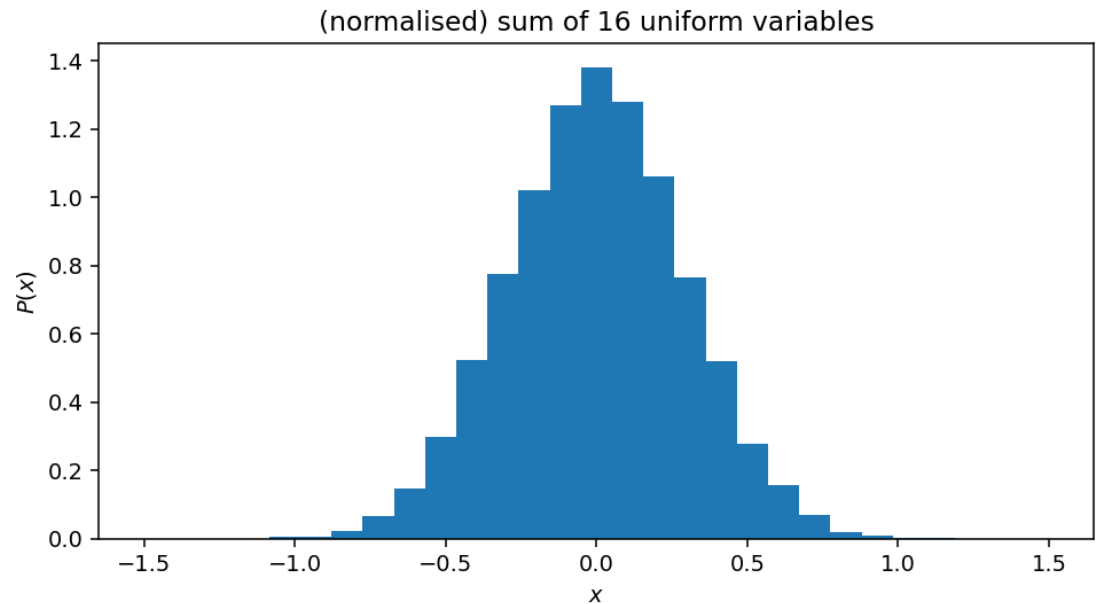




Central limit theorem

If we form a sum of many random variables $Y=X_1+X_2+X_3+\dots$, then for almost any PDF that each of $X_1, X_2, \dots$ might have, the PDF of Y will be approximately normal, $Y \sim N(\mu, \sigma^2)$.

This means that any process that involves a mixture of many random components will tend to be Gaussian under a wide variety of conditions.

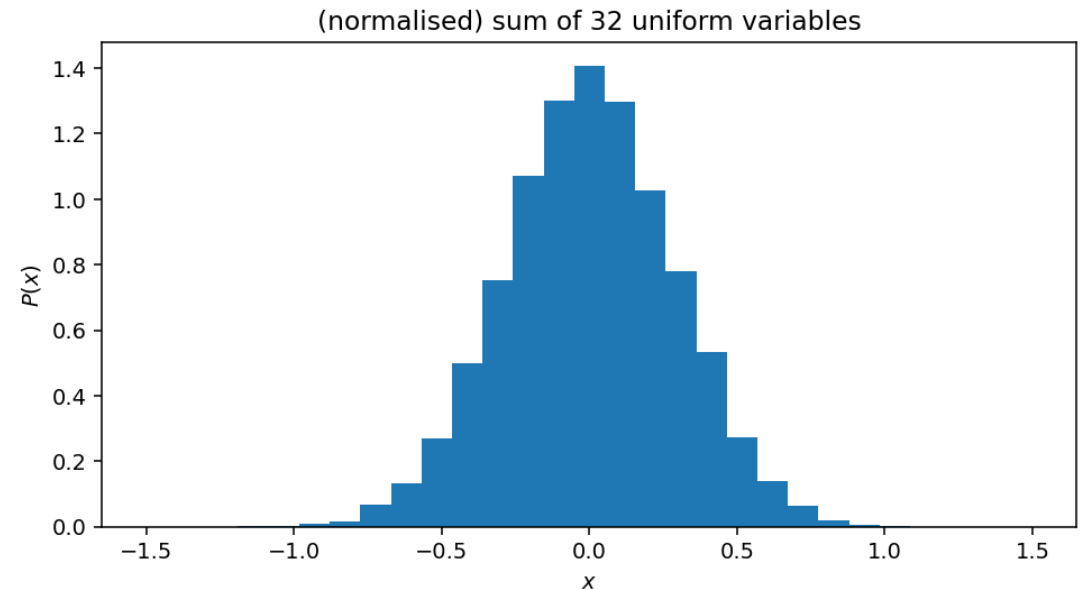




Central limit theorem

If we form a sum of many random variables $Y = X_1 + X_2 + X_3 + \dots$, then for almost any PDF that each of $X_1, X_2, \dots$ might have, the PDF of Y will be approximately normal, $Y \sim N(\mu, \sigma^2)$.

This means that any process that involves a mixture of many random components will tend to be Gaussian under a wide variety of conditions.



Multivariate distributions over $\mathbb{R}^n$

Continuous distributions generalise discrete variables to continuous spaces over $\mathbb{R}$ via probability density functions.

Probability densities can be further generalised to vector spaces, particularly to $\mathbb{R}^n$, under the constraint

$$\int_{\mathbf{x} \in \mathbb{R}^n} f_X(\mathbf{x}) = 1.$$

This is the same as:

$$\int_{x_0=-\infty}^{x_0=\infty} \int_{x_1=-\infty}^{x_1=\infty} \dots \int_{x_n=-\infty}^{x_n=\infty} f_X([x_0, x_1, \dots, x_n]) dx_0 dx_1 \dots dx_n = 1.$$

Distributions with PDFs over vector spaces are called **multivariate distributions**.

In many respects, they work the same as **univariate** continuous distributions. However, they typically require more parameters to specify their form, since they can vary over more dimensions.



Multivariate Uniform

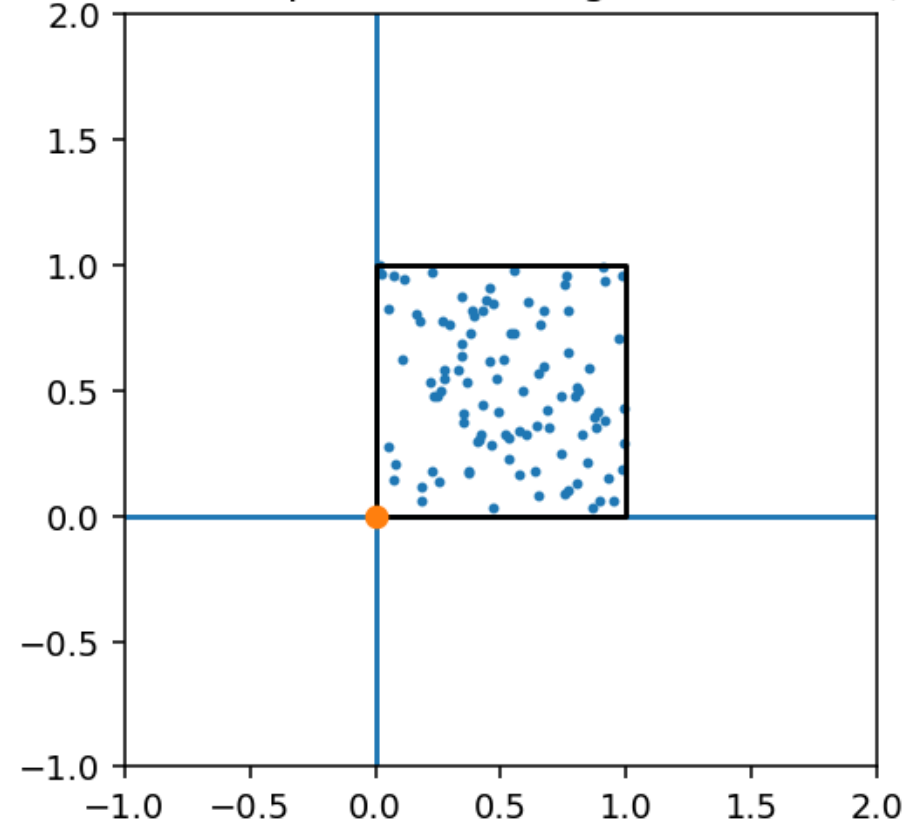
The multivariate uniform distribution is particularly simple to understand. It assigns equal density $f_{\mathbf{x}}(\mathbf{x}_i) = f_{\mathbf{x}}(\mathbf{x}_j)$ to some (axis-aligned) box in a vector space $\mathbb{R}^n$, such that

$$\int_{\mathbf{x}} f_{\mathbf{x}}(\mathbf{x}) = 1, \mathbf{x} \in \text{a box}.$$

It is trivial to sample from; we just sample *independently* from a one-dimensional uniform distribution in the range $[0,1]$ to get each element of our vector sample.

This is a draw from a n -dimensional uniform distribution in the unit box.

Uniform samples in the range $\mathbf{x} \in \mathbb{R}^2, 0 \leq x_i \leq 1$





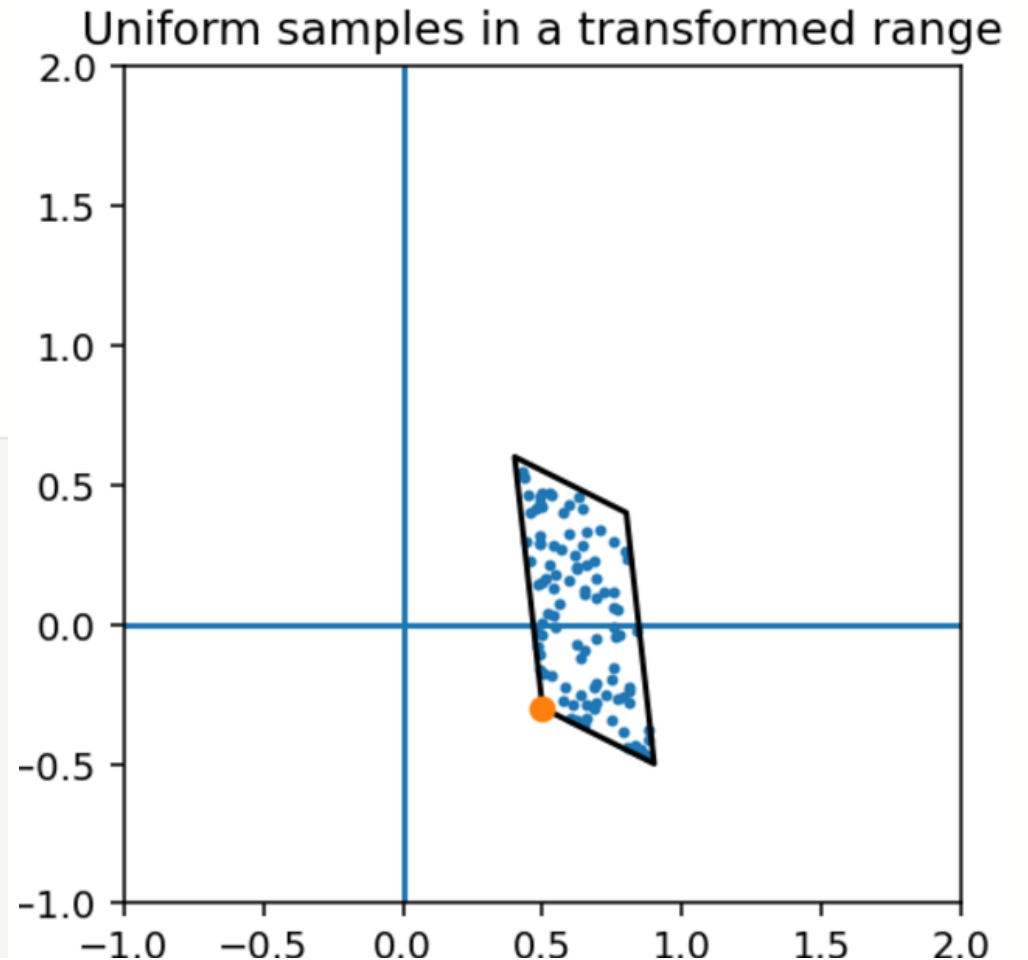
Transformed uniform distribution

If we want to define a distribution over any box, we can simply transform the vectors with a matrix A and shift by adding an offset vector b

```
[9]: # A is "spread"
A = np.array([[0.4, -0.2],
              [-0.1, 0.9]])

# b is "offset" or "centre"
b = np.array([0.5, -0.3])

pts, A, b = uniform_nd(100, A=A, b=b, d=2)
draw_pts_2d(pts, A=A, b=b)
_ = plt.gca().set_title("Uniform samples in a transformed range")
plt.show()
```





Normal distribution

The normal distribution (above) is very widely used as the distribution of continuous random variables.

It can be defined for a random variable of *any dimension*; a **multivariate normal** in statistical terminology.

We saw the idea of a **mean vector** μ and a **covariance matrix** Σ which captured the "shape" of a dataset in terms of an ellipse. *These are in fact the parameterisation of the multivariate normal distribution.*



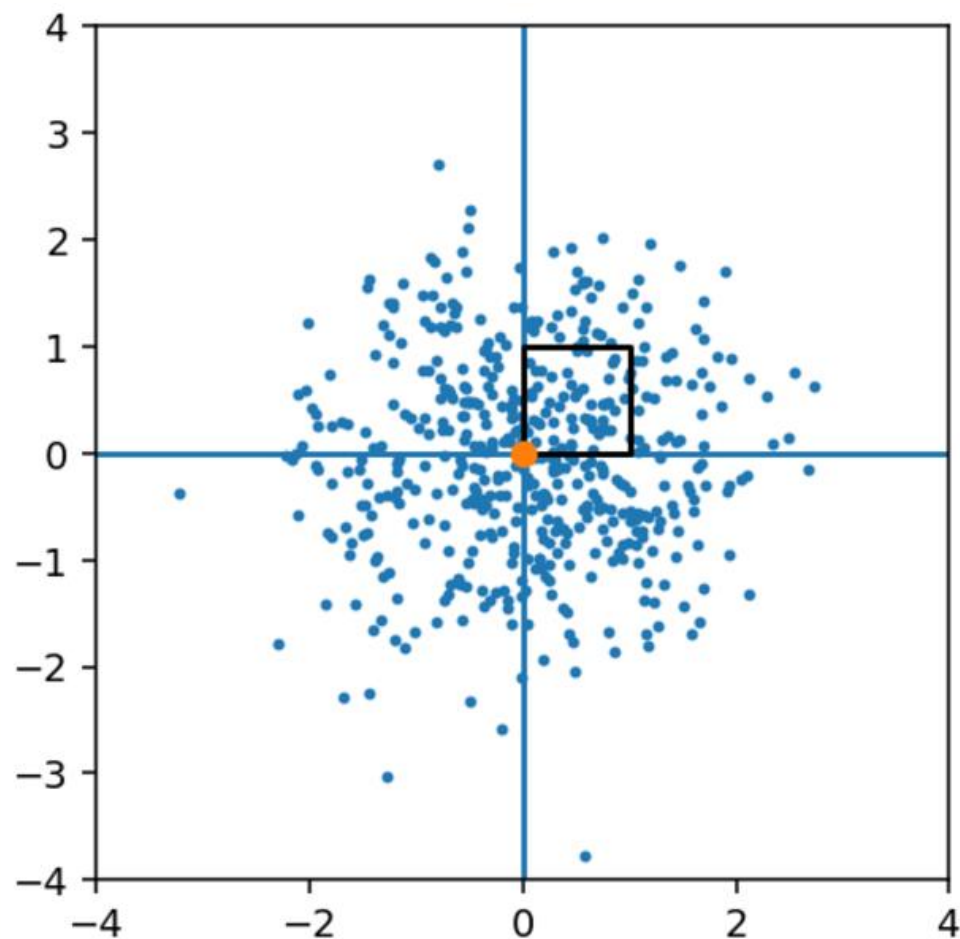
Normal distribution

- A multivariate normal is fully specified by a mean vector μ and the covariance matrix Σ . If you imagine the normal distribution to be a ball shaped mass in space, the mean *translates* the mass, and covariance applies a transformation matrix (scale, rotate and shear) to the ball.
- Just like the uniform distribution, we can think of drawing samples from a "unit ball" with an independent normal distribution in each dimension. These samples are transformed linearly by the covariance matrix Σ and the mean vector μ , just like A and b above (though Σ is actually $A^{-1}A^{-T}$ for technical reasons)



Normal distribution

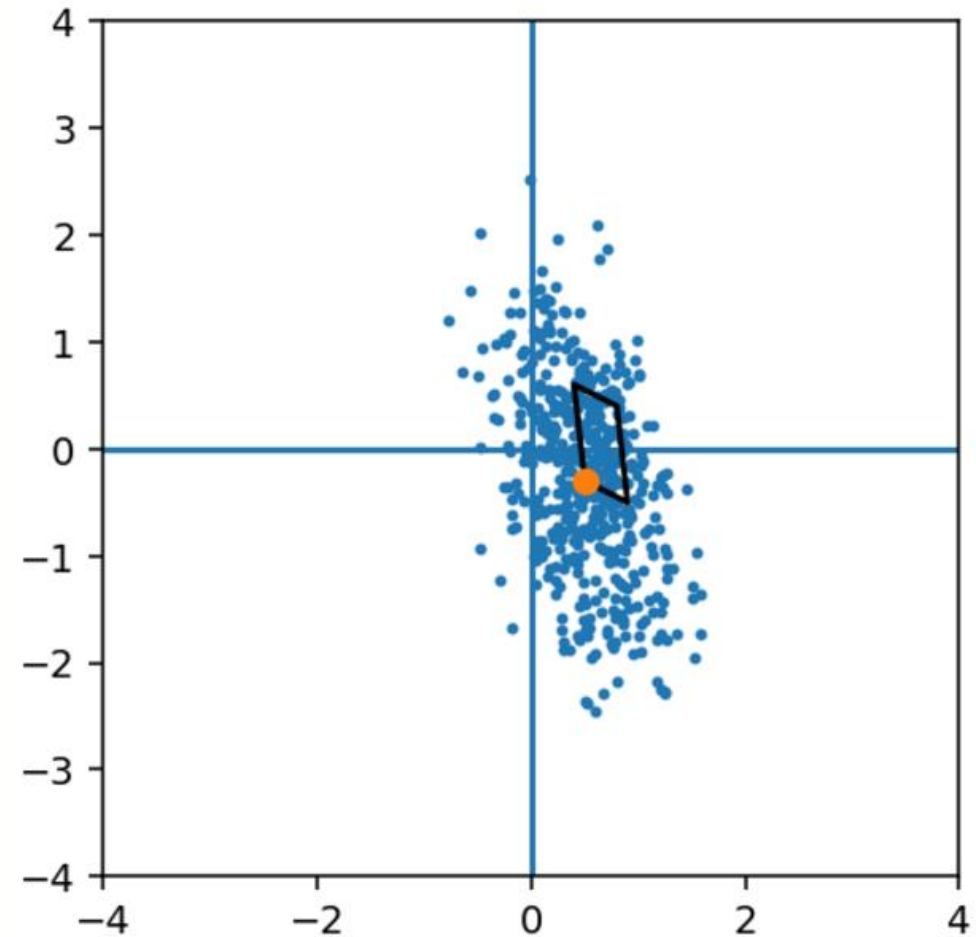
```
[10]: def normal_nd(n, A = None, b=None, d=2):  
    if A is None:  
        A = np.eye(d)  
    if b is None:  
        b = np.zeros(d)  
    return np.random.normal(0,1, (n,d)) @ A + b, A, b  
  
pts, A, b = normal_nd(500, A=None, b=None, d=2)  
draw_pts_2d(pts, A=A, b=b)  
plt.gca().set_xlim(-4,4)  
_ = plt.gca().set_ylim(-4,4)  
plt.show()
```





Normal distribution

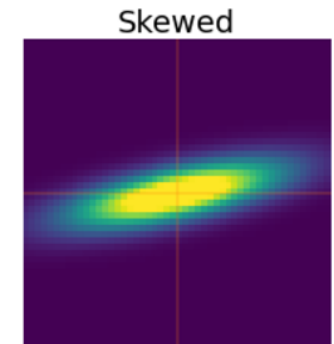
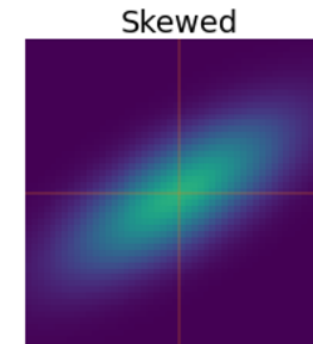
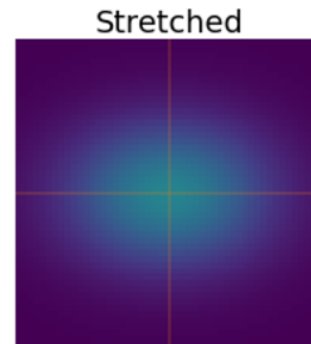
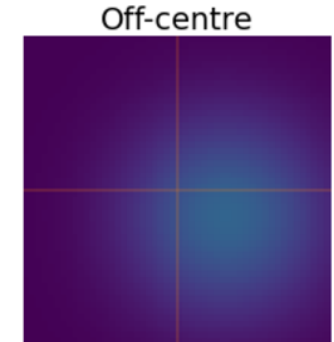
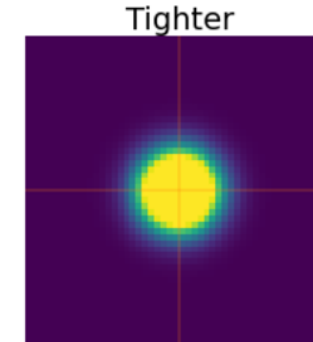
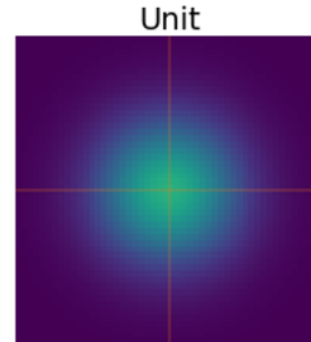
```
[11]: # A is "spread"
A = np.array([[0.4, -0.2], [-0.1, 0.9]])
# b is "offset" or "centre"
b = np.array([0.5, -0.3])
pts, A, b = normal_nd(500, A=A, b=b, d=2)
draw_pts_2d(pts, A=A, b=b)
plt.gca().set_xlim(-4,4)
_ = plt.gca().set_ylim(-4,4)
plt.show()
```





Joint and marginal PDFs

We can look at the PDF of a multivariate normal for different covariances and mean vector (centres and spreads).





Joint PDF

We can now talk about the **joint probability density function** (density over all dimensions) and the **marginal probability density function** (density over some sub-selection of dimensions).

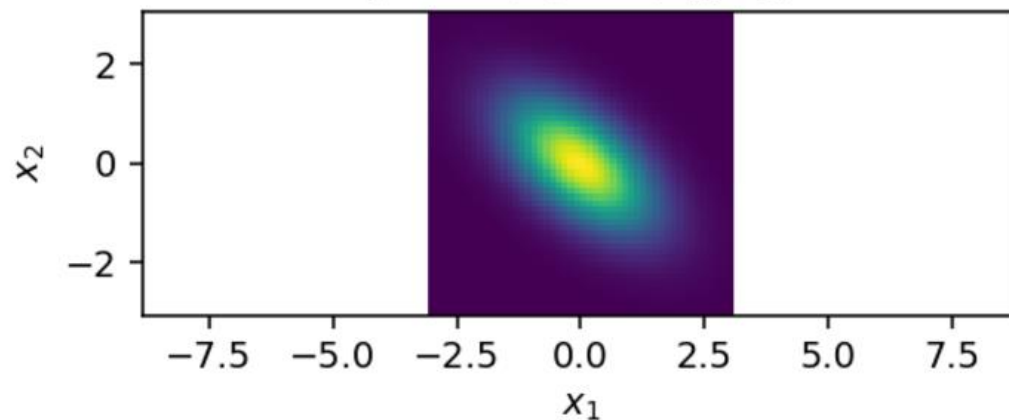
For example, consider $X \sim N(\mu, \Sigma)$, $X \in \mathbb{R}^2$, a two dimensional ("bivariate") normal distribution. We can look at some examples of the PDF, showing:

- Joint $P(X)$
- Marginal $P(X_1)$ and $P(X_2)$
- Conditionals $P(X_1 | X_2)$ and

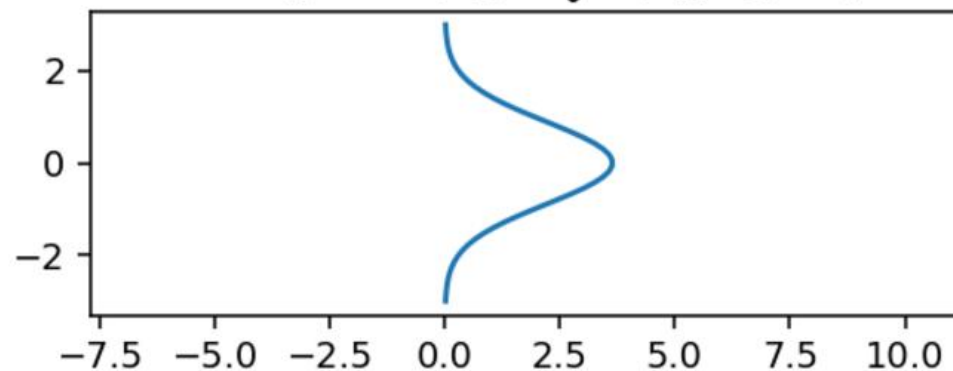


Marginal PDF

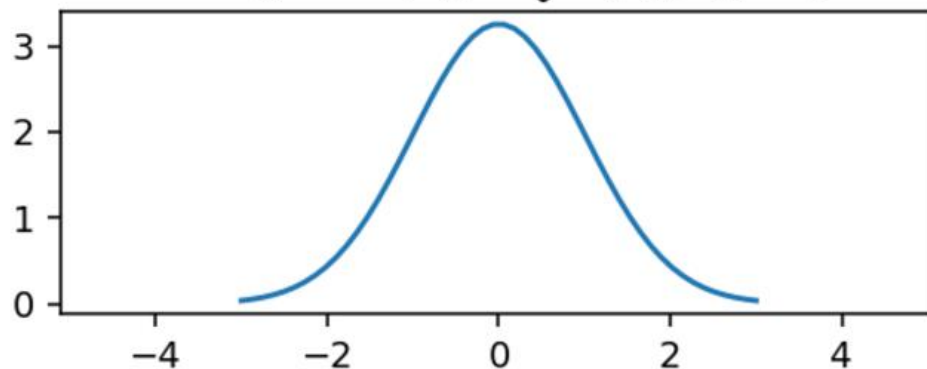
Joint $P(\mathbf{x}) = P(x_1, x_2)$



Marginal $P(x_2) = \int P(x_1, x_2) dx_1$



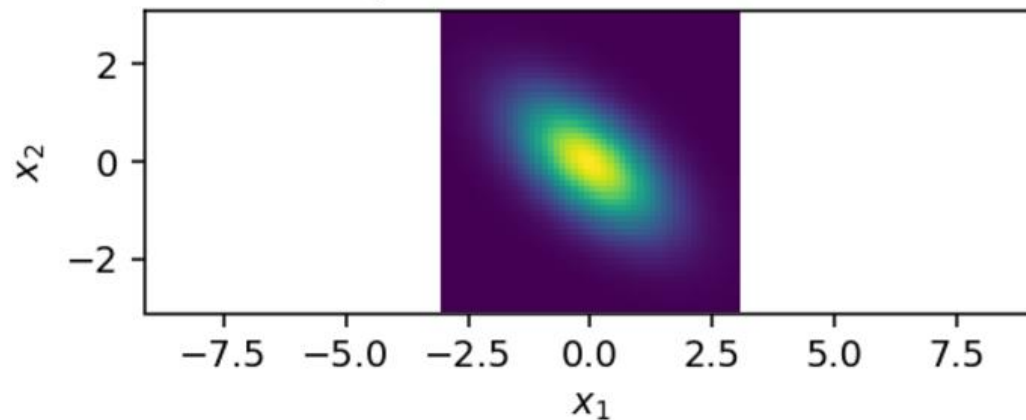
Marginal $P(x_1) = \int P(x_1, x_2) dx_2$



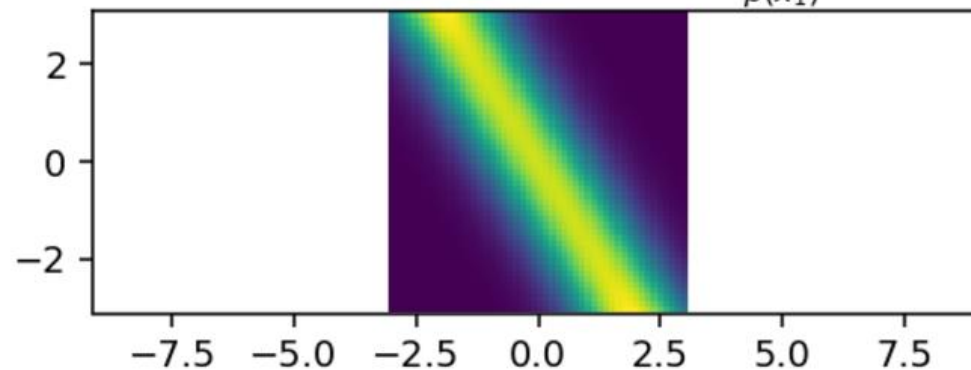


Conditional PDF

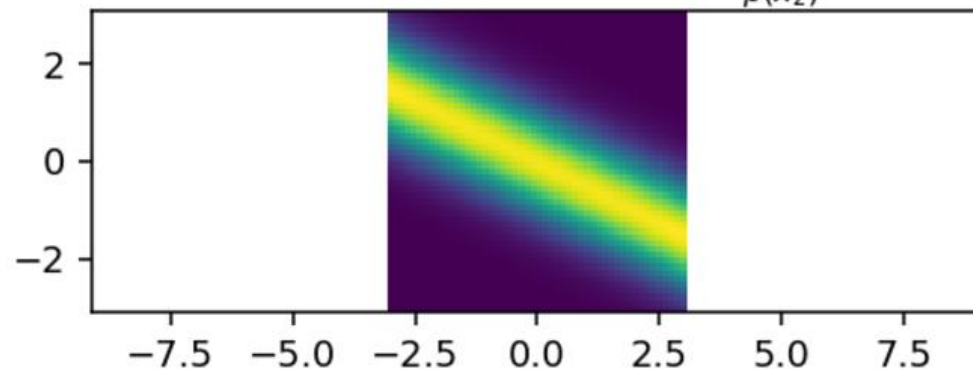
Joint $P(\mathbf{x}) = P(x_1, x_2)$



Conditional $P(x_2|x_1) = \frac{P(x_1, x_2)}{p(x_1)}$



Conditional $P(x_1|x_2) = \frac{P(x_1, x_2)}{p(x_2)}$





How do we *draw* samples from a continuous distribution?

How can we simulate the outcomes of a random variable X ?

This is a vital tool in computational statistics. One of the reasons computers are useful for statistical analysis is that they can generate (pseudo)-random numbers very quickly.



Monte Carlo

- During the *Manhattan project* that developed the atomic bomb during the Second World War, there were many difficult probabilistic equations to work out.
- Although *analytical techniques* for solving certain kinds of problems existed, they were only effective some narrow types of problem and were tricky to apply to the problems that the Manhattan project had to solve.





Monte Carlo

John von Neumann and Stanislaw Ulam developed the **Monte Carlo** method to approximate the answer to probabilistic problems, named after the casinos of Monte Carlo.

This involved setting up a *simulation* with stochastic (random) components.

By running the simulation many times with different random behaviour, the population of *possible* behaviours could be approximated.



Stanislaw Ulam



John von Neumann

Monte Carlo

For example, computing the expectation of a function of a random variable can often be hard for continuous random variables.

The integral for:

$$\mathbb{E}[g(X)] = \int_{\mathbf{x}} f_X(\mathbf{x})g(\mathbf{x}) d\mathbf{x}$$

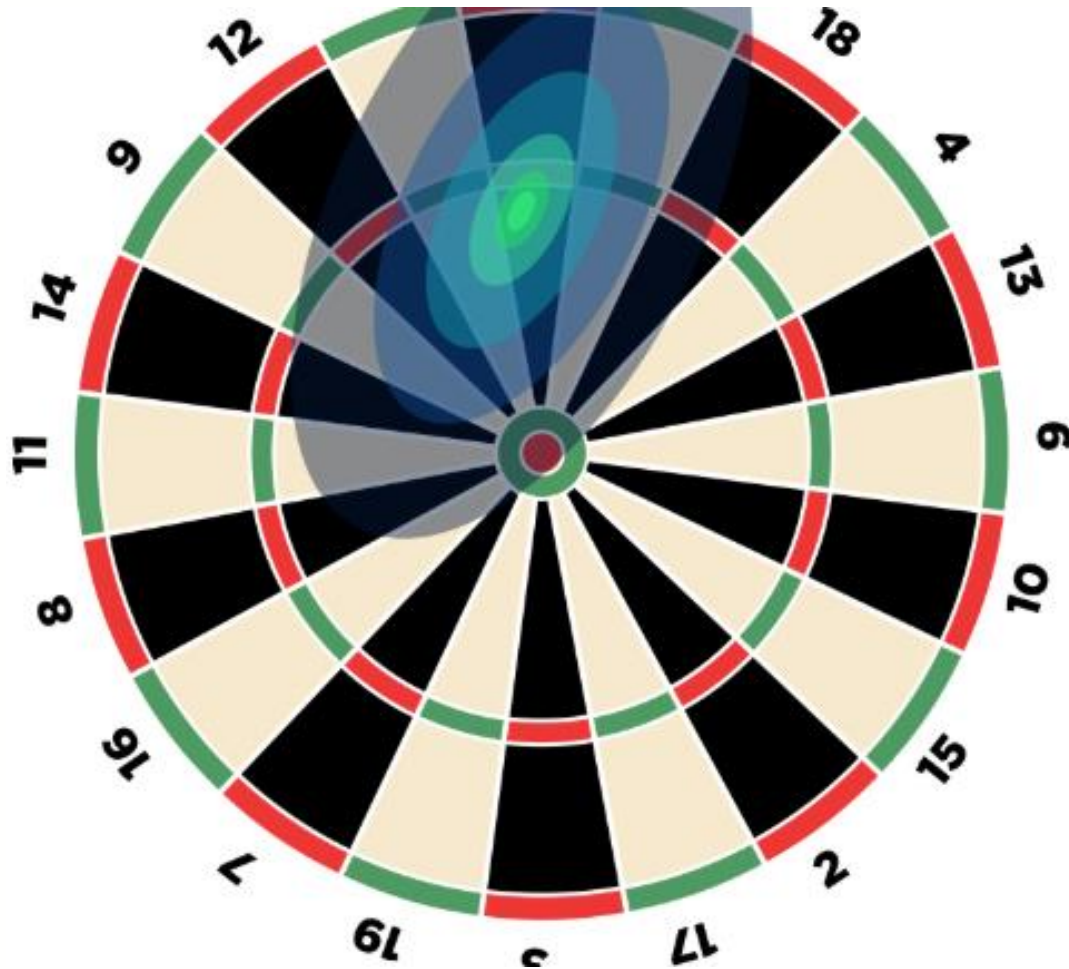
may be intractable. However it is often very easy to compute $g(\mathbf{x})$ for any possible $\mathbf{x}$. If we can somehow sample from the distribution $P(X=\mathbf{x})$, then we can approximate this very easily:

$$\mathbb{E}[g(X)] = \int_{\mathbf{x}} f_X(\mathbf{x})g(\mathbf{x}) d\mathbf{x} \approx \frac{1}{N} \sum_{i=1}^N g(\mathbf{x}_i) ,$$

where $\mathbf{x}_i$ are random samples from $P(X=\mathbf{x})$, defined by the PDF $f_X(\mathbf{x})$. This gets better as N gets larger.



Throwing darts



For example, imagine trying to work out the expectation of dart throw. A dart board has sections giving different scores.

We might model the position of the dart as a normal distribution over the dart space. This models the human variability in throwing.

The expected score of a throw requires evaluating the integral of the normal PDF multiplied by the score at each point -- which isn't feasible to compute directly.



Throwing darts



But we can sample from a multivariate normal distribution easily; we saw this in the last unit; just sample from d independent standard normals, and transform with a linear transform (matrix).

So instead of trying to solve a very hard integral, we can simulate lots of dart throws, which follow the pattern of the normal distribution, and take the average score that they get.

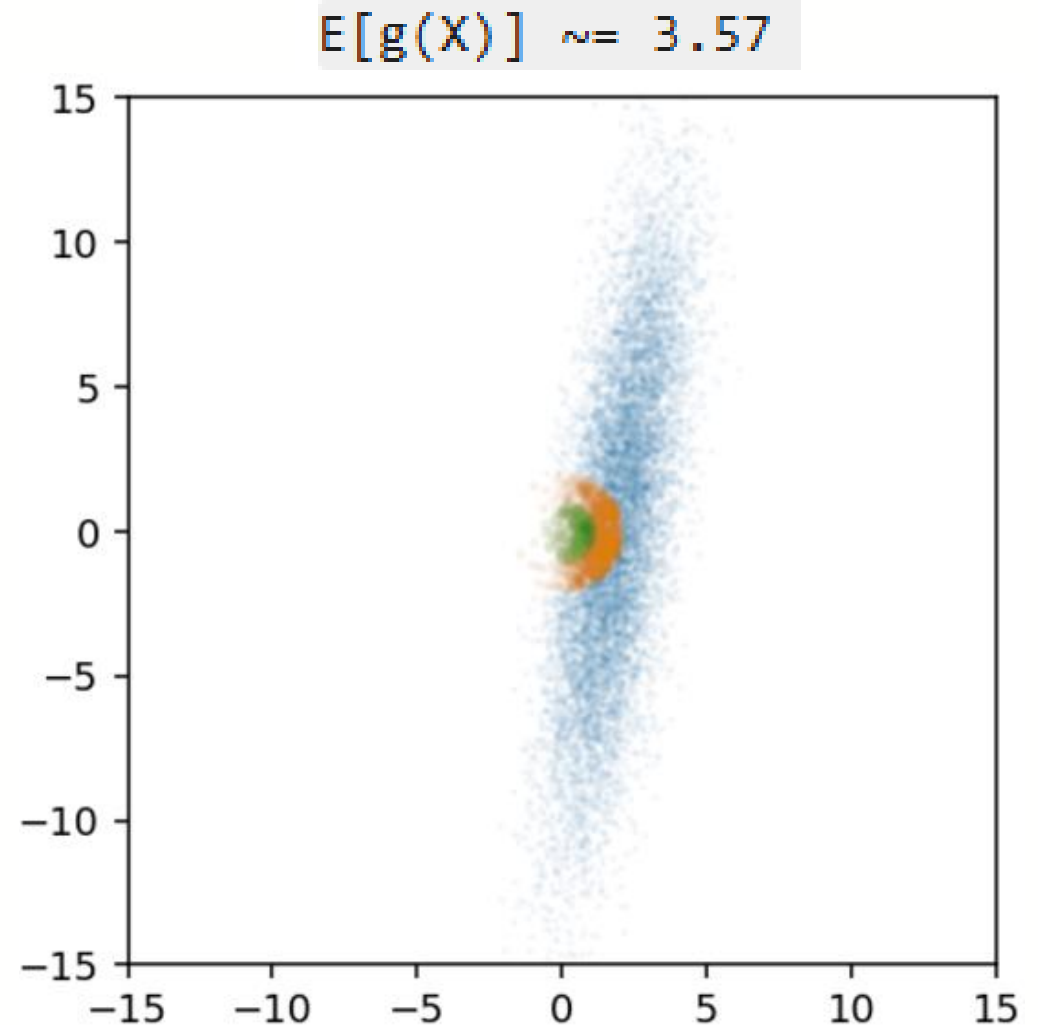
If we simulate a lot of darts, the average will be close to the true value of the integral.



Bullseye example

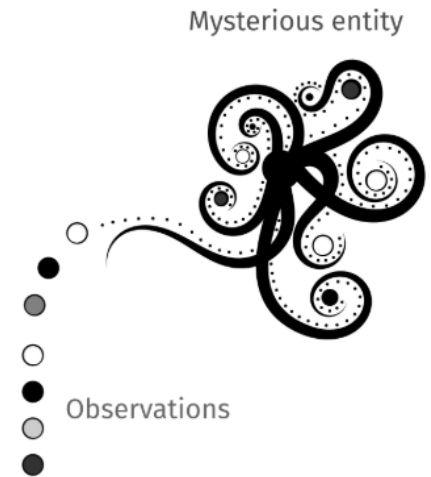
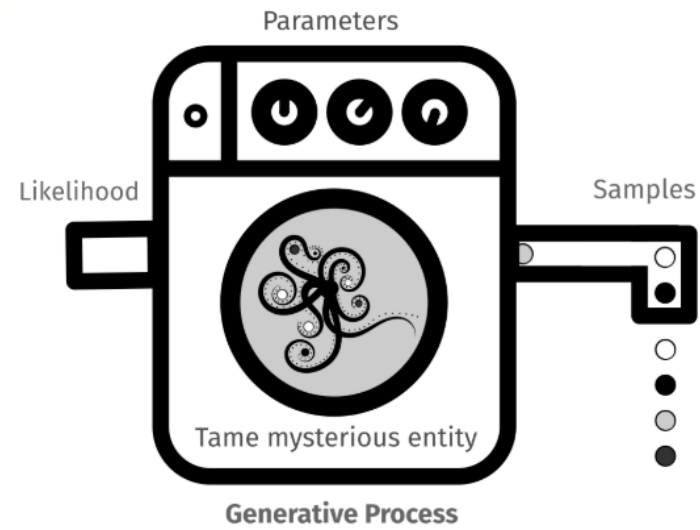
For example, we might want to define a circular score region, which gives us 25 points if we land in it, 50 points if we lie in a smaller coencentric circle, and 0 otherwise; this is our function $g(X)$. We might model the throw of the dart with a multivariate normal.

How do we compute the expected score $E[g(X)]$?





Inference





Inference

- **Inferential statistics** is concerned with estimating the properties of an unobserved "global" **population** of values from a limited set of observed **samples**.
- This assumes that there is some underlying distribution from which samples are being drawn.
- This is a hidden process (the "mysterious entity"), which we only partially observe through the samples we see.



Population is the unknown set of outcomes (which might be infinite)

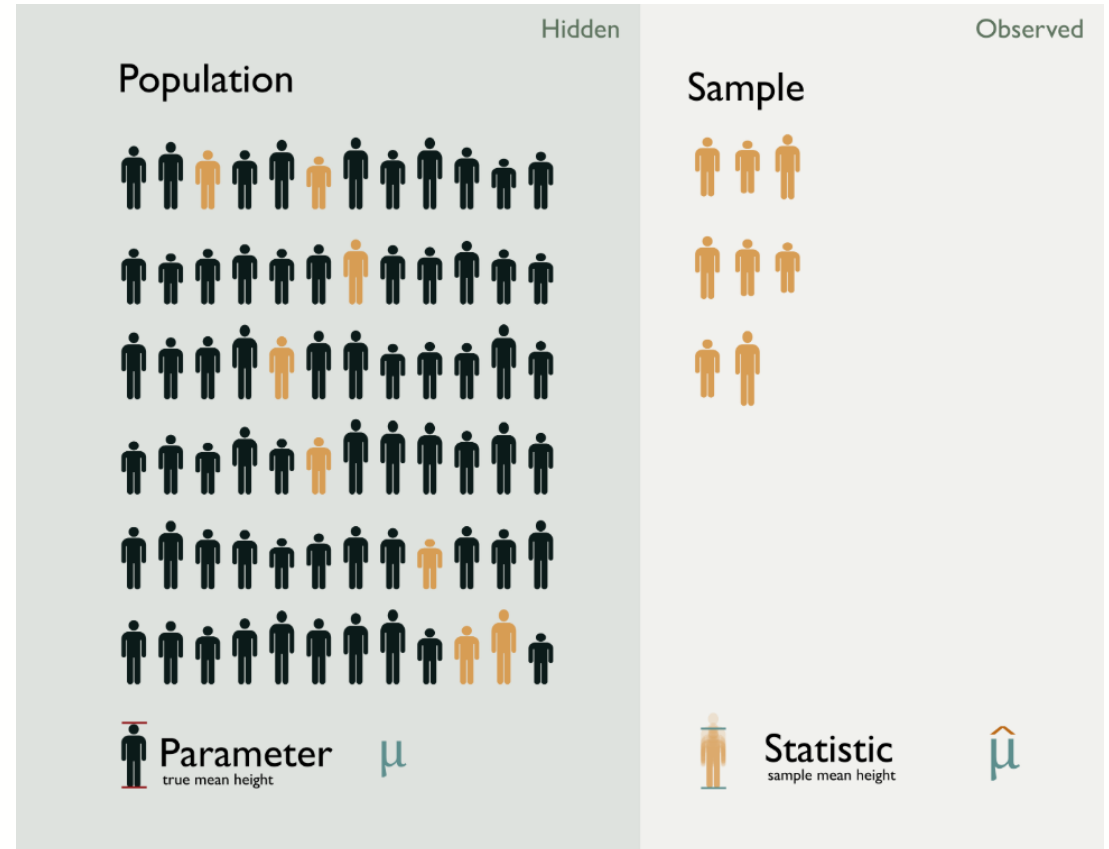
- **Example** the weight of all beetles
- **Parameter** describes this **whole population**, e.g. the mean weight of all beetles

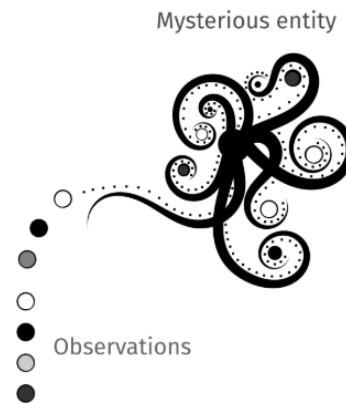
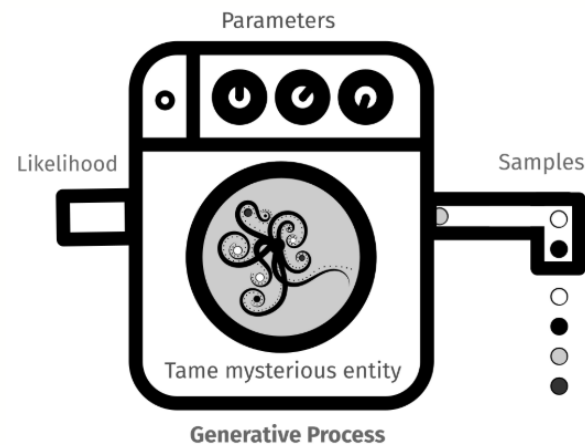
Sample is some subset of the population that has been observed.

- **Example** 20 beetles whose weight has been measured
- **Statistic** is a function of the sample data, e.g. the arithmetic mean of those 20 samples

The parameters of the population distribution govern the generation of the samples that are observed.

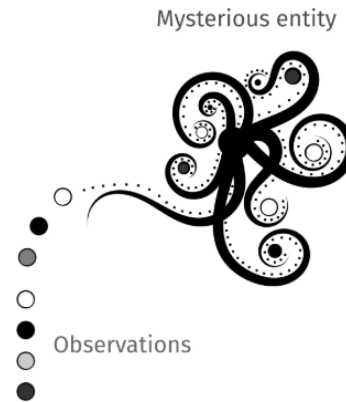
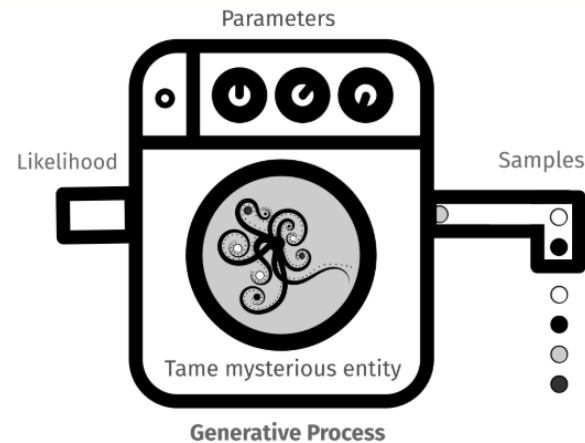
The problem of statistics is how to **infer** parameters given samples.





Our model of the world is that there is some unknown entity which **generates** data that we observe, according to some definite but unknown rules.

These rules are codified by a **distribution** (a "type" of rule) and **parameters** (the specifics of rules applied). We assume the model has some *randomness* or *stochastic* elements, either because it truly does, or because this makes it simpler to represent the model.



Inference is the process of determining these rules (i.e. the parameters) by looking at the aftermath of the actions of the mysterious entity.

These are the **samples** or **observations** that we have. From these we can work out what must have been going on in the mysterious entities world. Or at least approximate it as well as we can.

We usually assume that we know or have chosen a specific **distribution** which we expect to be governing the process, and focus on identifying the parameters involved.



Estimator

Look at observations
Algorithmically adjust parameters
Adjust process to match **directly**

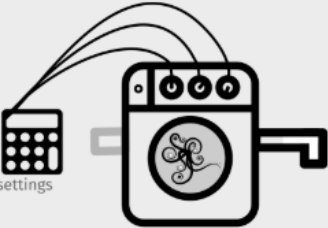
Direct estimation

$$\hat{\mu} \Rightarrow \mu$$

Observations

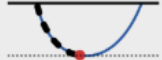


Calculate knob settings



Maximum Likelihood

Compute likelihood of each observation
Tweak knobs to maximise likelihood
Optimise likelihood (usually negative log likelihood is easier)
Log-likelihood optimisation

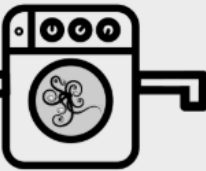


Observations



Likelihood

Twiddle knobs



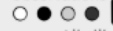
Bayesian Inference

Form guesses over possible knob states
Update guesses based on likelihood
Result in **distributions** over knob settings

Bayes' Rule

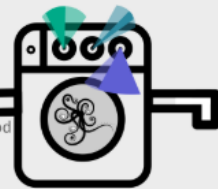
$$P(A|B) \propto P(B|A)P(A)$$

Observations



Likelihood

Update knob beliefs
Prior -> Posterior



We will see three different approaches to doing inference:

- **Direct estimation** of parameters, where we define *functions of observations* that will estimate the values of parameters of distributions *directly*.
- **Maximum likelihood estimation** of parameters, where we use **optimisation** to find parameter settings that make the the observations appear as likely as possible.
- **Bayesian, probabilistic** approaches explicitly encode belief about the behaviour of the mysterious entity using probability distributions. In Bayesian models, we assume a distribution over the parameters themselves, and consider the *parameters to be random variables*.



University
of Glasgow

An inference scenario: The app rating problem



You've written an app, and it'll make you rich. If you can make a version that people really like, that is.

So maybe you've released a few different beta version to try out some options. Each user has rated the app with 1-5 stars.

You need to work out which version is better.

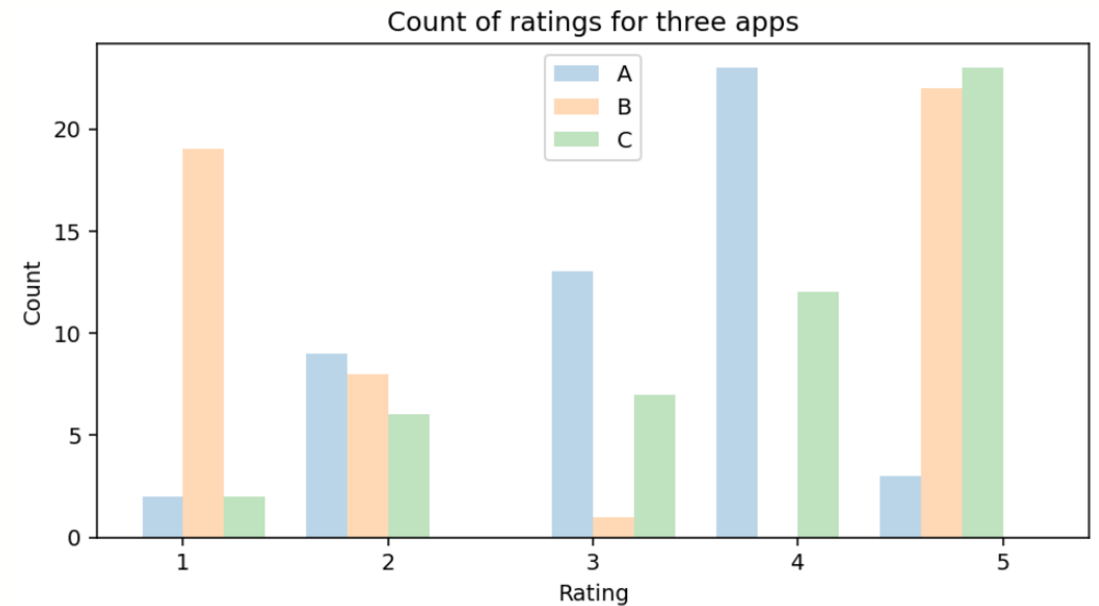


An inference scenario: The app rating problem

The problem is that you only have a sample of responses.

Not every user rated the app, and actually you don't even care about the users who have *already* bought the app and rated it.

You want to know how *prospective* customers will view it, and by definition you cannot have sampled from this population of users.



An inference scenario: The app rating problem

Imagine we are trying to infer the distribution of app scores *assuming that they were generated by a normal distribution*. That is, we imagine there is some function like this:

```
[17]: def app_samples(mu, sigma):  
      return np.random.normal(mu, sigma)
```

This isn't a very good approximation to the real samples; for example, it can generate negative ratings, or ratings with fractional values. It also assumes that all of the ratings that we see are independent of each other (**independence assumption**) and that they are all drawn the *same* underlying distribution (**identical distribution assumption**).

But it is simple to work with, and the problem we have to solve is: given a collection of return values from this function (samples), what values did μ and σ have?

This is a problem of inference



A. Direct estimates

Estimator

Look at observations

Algorithmically adjust parameters

Adjust process to match **directly**

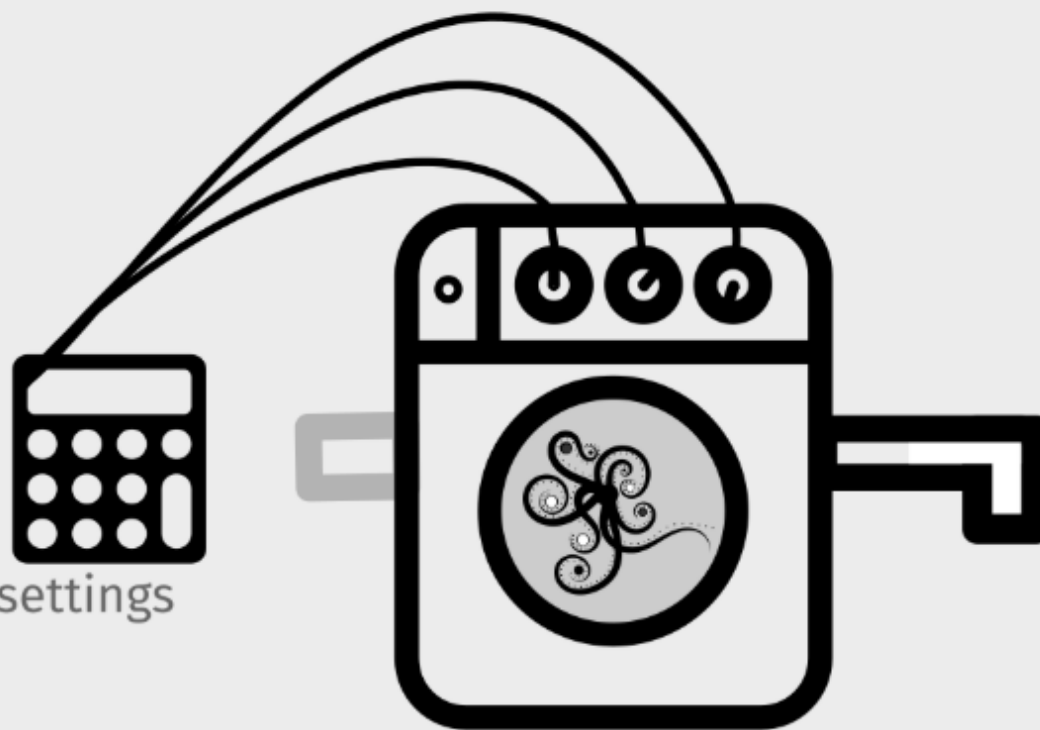
Direct estimation

$$\hat{\mu} \Rightarrow \mu$$

Observations



Calculate knob settings





Direct estimation

One way of doing inference is to, if we assume a particular *form* of the distribution (e.g. assume it is normal), use **estimators** of **parameters** (such as the mean and variance) of this population distribution.

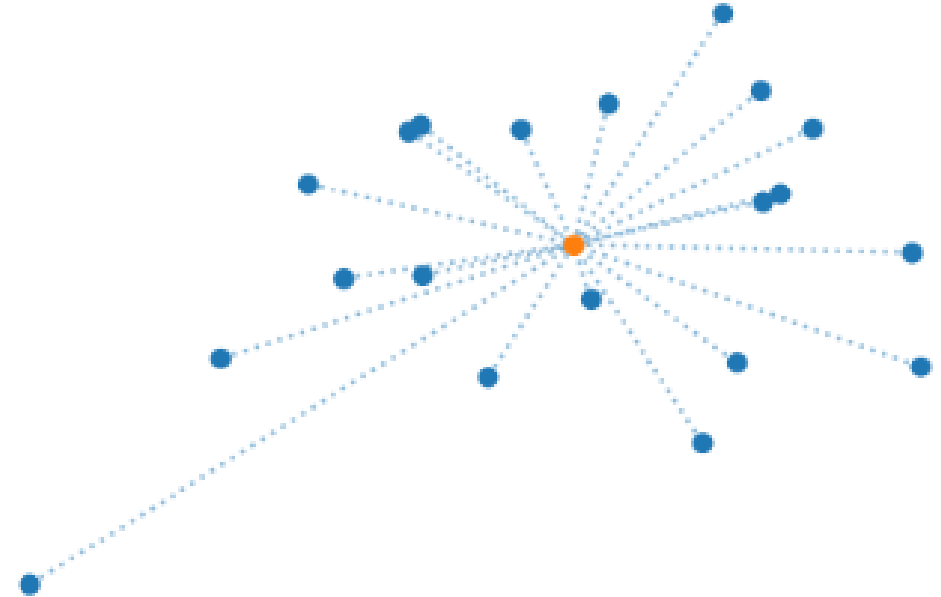
These **estimators** are computed via **statistics** which are summarising functions we can apply to data.

These estimators need to specially derived for each specific kind of problem.



Example: mean and variance

- For example, the arithmetic mean, and the standard deviation of a set of observed samples are **statistics** which are **estimators** of the parameters of μ and σ normal distribution.
- If we have observations (believed to have been) drawn from a normal distribution, we can estimate the parameters μ and σ of that distribution just by computing the mean and standard deviation.





Standard estimators

Mean

The **arithmetic mean** is sum of sample values $x_1, x_2, \dots, x_n$ divided by the number of values:

$$\hat{\mu} = \frac{1}{N} \sum_{i=1}^N x_i$$





Sample mean

The population mean is $\mu = E[X]$ for a random variable X . It turns out the *arithmetic mean of the observed samples* or **sample mean**, which we write with a *little hat* $\mu^\wedge$ is a good (footnote: good is what statisticians would call "unbiased") estimator of the true population mean μ . As the number of samples increases, our estimate $\mu^\wedge$ of the population mean μ gets better and better.

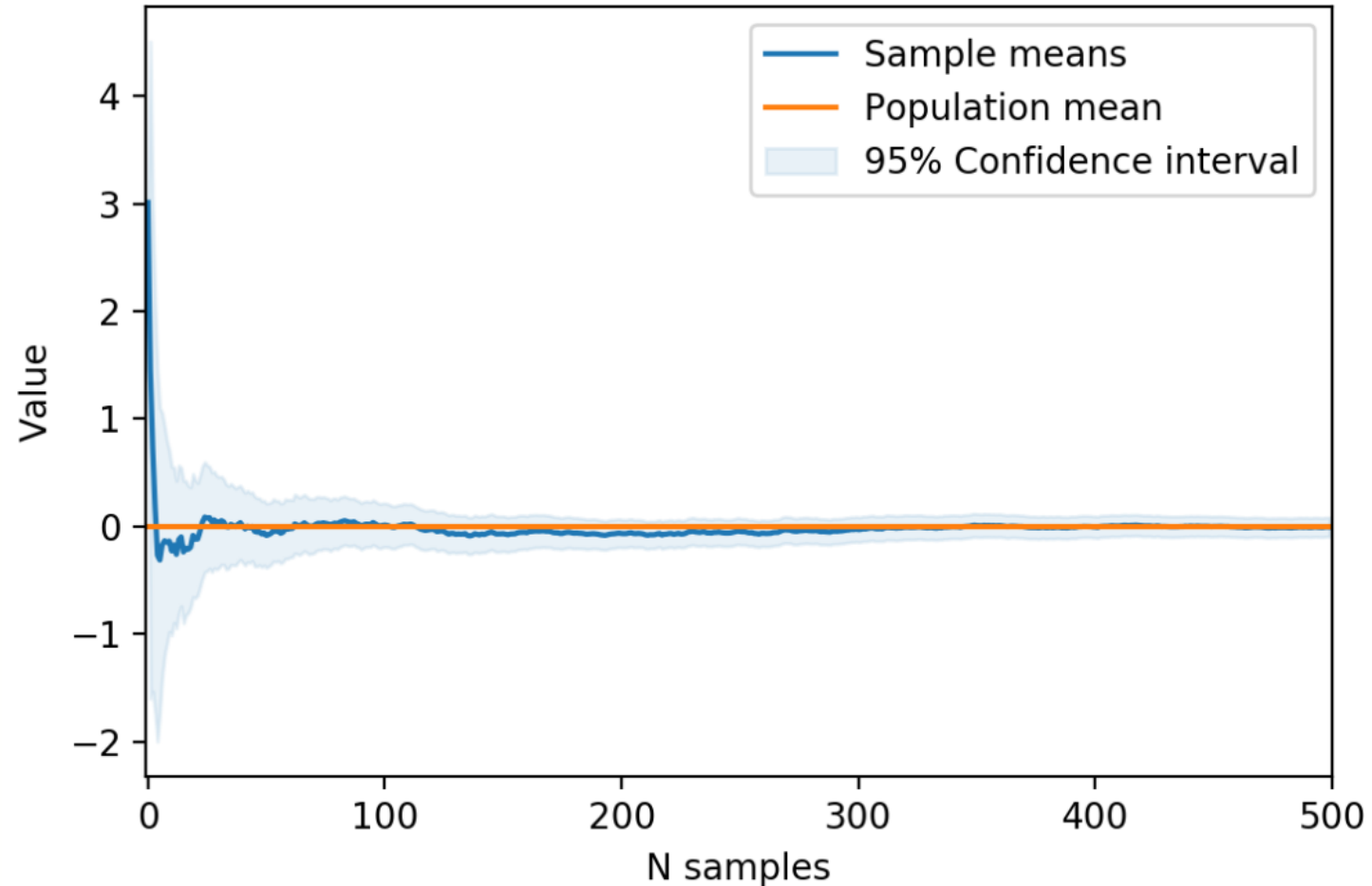
It's important to separate the idea of

- the population mean μ , which (usually!) exists but is not knowable directly. It is the expectation of the random variable $E[X]$.
- the sample mean $\mu^\wedge$ which is just the arithmetic average of samples we have seen

The sample mean is a **statistic** (a function of observations) which is an **estimator** of the population mean (which could be a **parameter** of a distribution). Specific bounds can be put on this estimate; the standard error gives a measure of how close we expect that the arithmetic mean of samples is to the population mean, although the interpretation is not straightforward.



Sample mean example



The mean measures the **central tendency** of a collection of values. The **mean vector** generalises this to higher dimensions.

Variance and standard deviation

The sample variance is the squared difference of each value of a sequence from the mean of that sequence:

$$\hat{\sigma}^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu})^2.$$

It is an estimator of the population variance, $E[(X - E[X])^2]$

The sample standard deviation is just the square root of this value.

$$\hat{\sigma} = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu})^2}$$

The variance and the standard deviation measure the **spread** of a collection of values.
The **covariance matrix** Σ generalises this idea to higher dimensions.

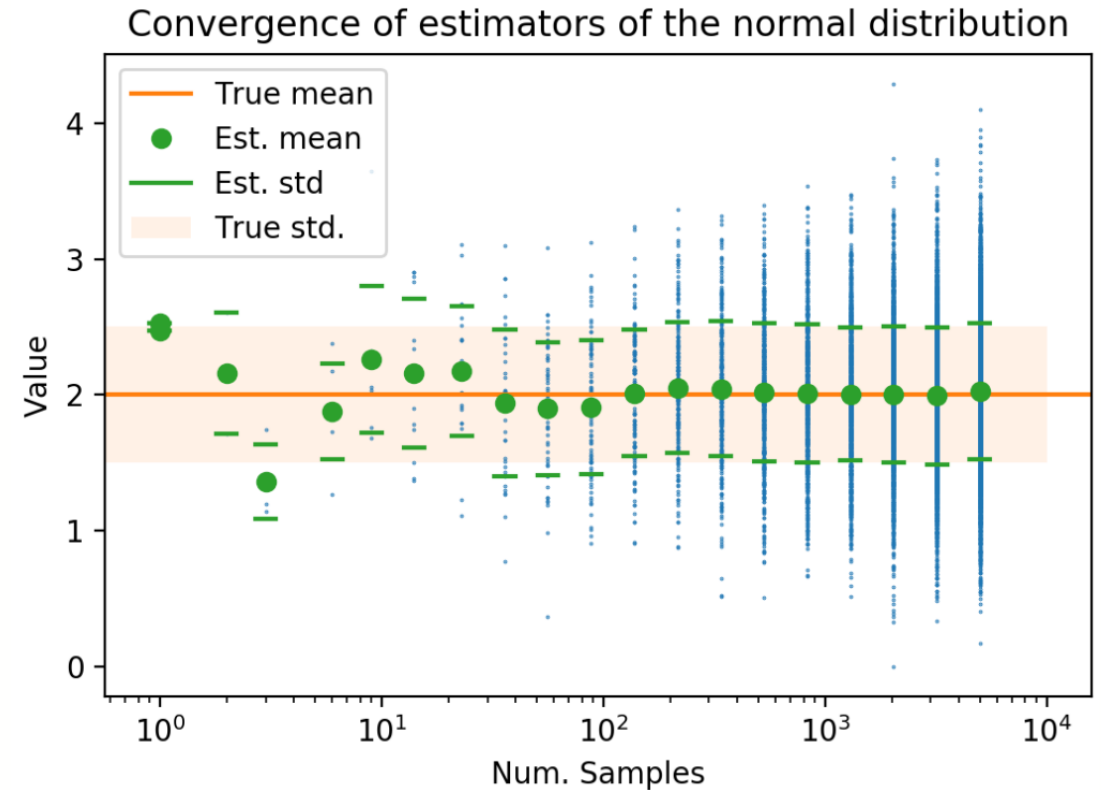


Relation to normal distribution

If we *assume* that our data is generated by a normal distribution, then the statistics **mean** $\mu^{\wedge}$ and **variance** $\sigma^{2\wedge}$ estimate the parameters μ, σ of that normal distribution, $N(\mu, \sigma)$.

Even if the underlying process isn't exactly normal, it may well be close to being normal because of the Central Limit Theorem.

And even if that doesn't apply, the mean and the variance are still useful *descriptive statistics*.



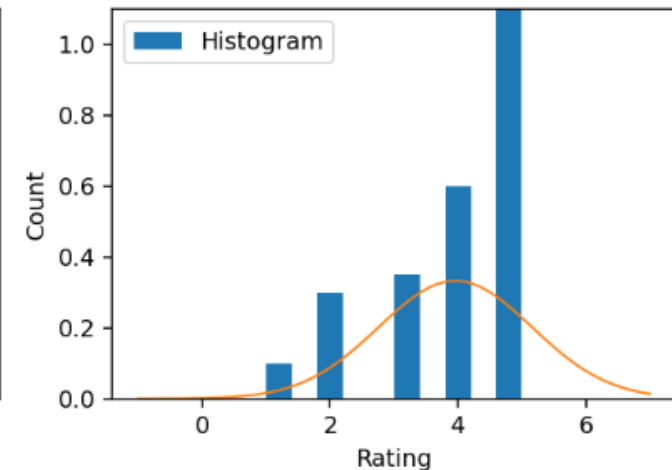
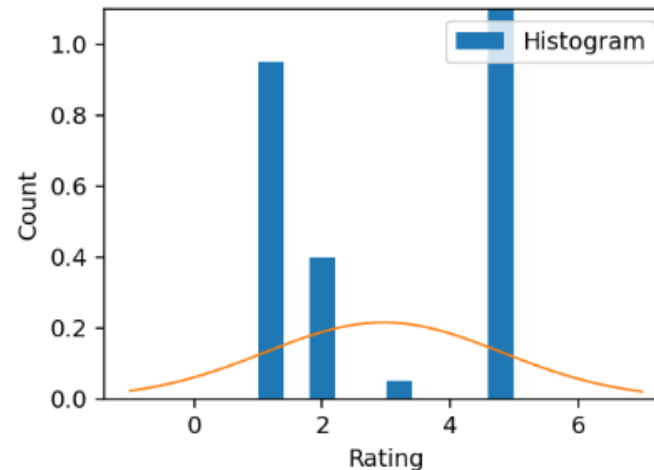
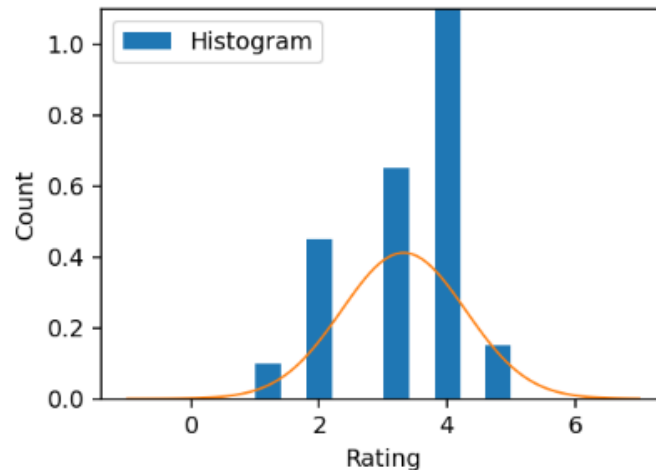


Fitting

What does it mean to estimate the parameters of a normal distribution that might be creating app ratings?

We are **fitting** a distribution, governed by a PDF, to a set of observations. But estimating a PDF requires some structure, a space of functions with some parameterisation.

We can visualise this:

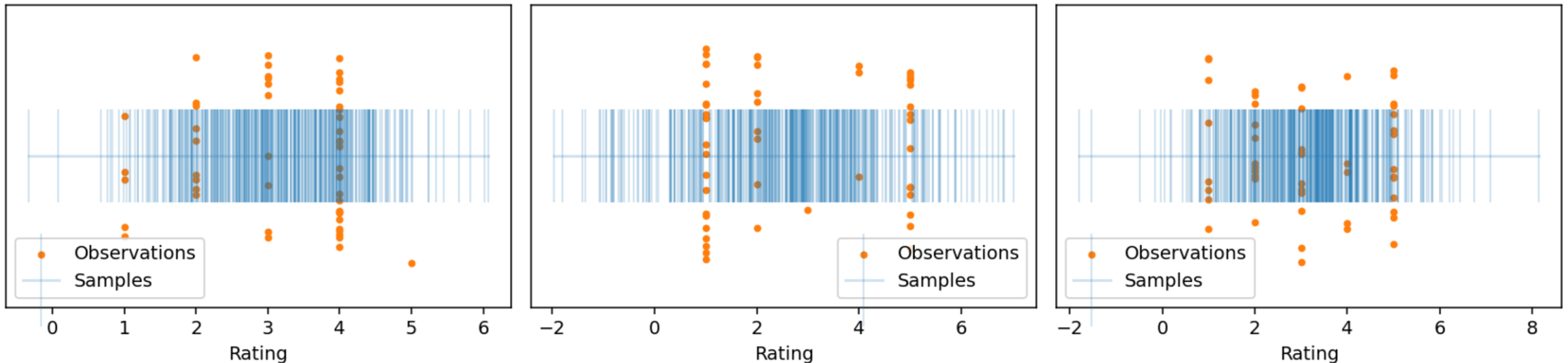




Sampling from the model

We can draw samples from our fitted distribution, and compare them to our results. They won't be a very good representation, because the data we have is clearly not normal.

But they show what our tame mysterious entity is producing, and let us assess our **modelling assumptions** -- that the app ratings were characterised by just a mean and standard deviation.





B. Maximum Likelihood

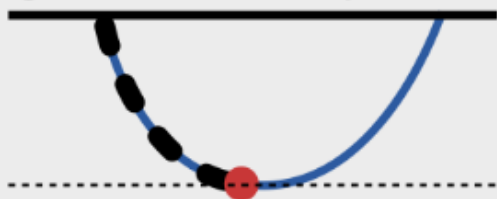
Maximum Likelihood

Compute likelihood of each observation

Tweak knobs to maximise likelihood

Optimise likelihood (usually negative
log likelihood is easier)

Log-likelihood optimisation

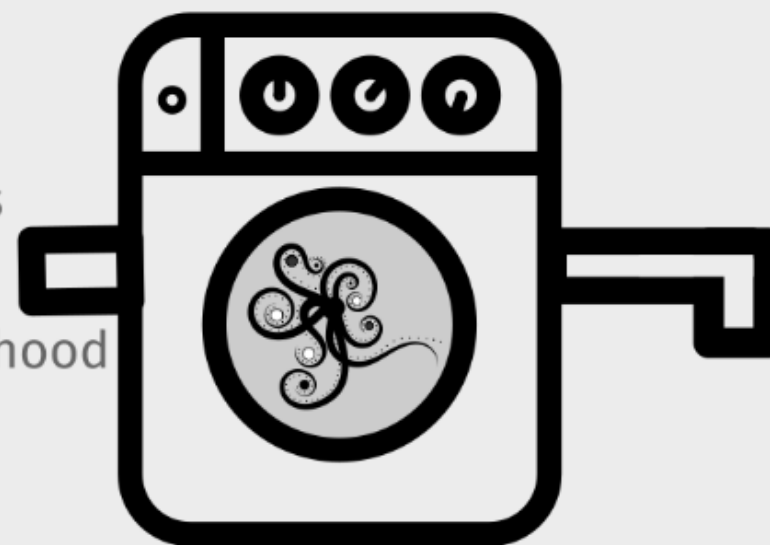


Observations



Likelihood

Twiddle knobs





Maximum likelihood estimation: estimation by optimisation

- What if we don't have estimators, ready built to estimate the parameters that we want? How can we do inference? How can we fit distribution parameters to observations?
- In many cases, we can compute the **likelihood** of an observation being generated by a specific underlying random distribution. This is the **likelihood** that we saw earlier.
- For a PDF, the likelihood of a value x is just the value of the PDF at x : $f_x(x)$.
- The likelihood is a function of the data, under the assumption of some particular parameters.

Maximum likelihood estimation: estimation by optimisation

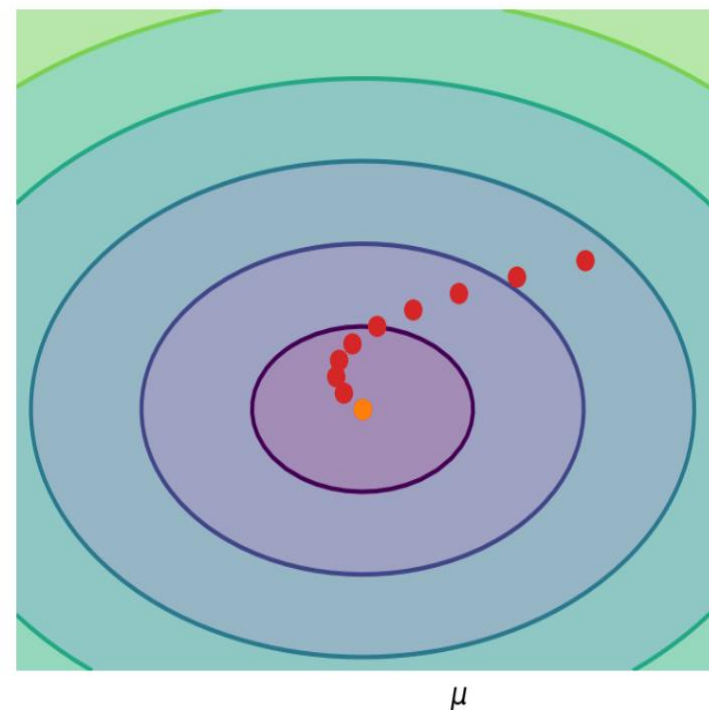
The likelihood of many *independent* observations is the product of the individual likelihoods, and the log-likelihood is the sum of the individual log-likelihoods.

$$\log \mathcal{L}(x_1, \dots, x_n) = \sum_i \log f_X(x_i)$$

Imagine we have a distribution which we *don't* know any **estimators** for the parameters.

How could we estimate what they might be, given some data?

We could write all of our parameters as vector θ ; for example a normal distribution would have $\theta = [\mu, \sigma]$.



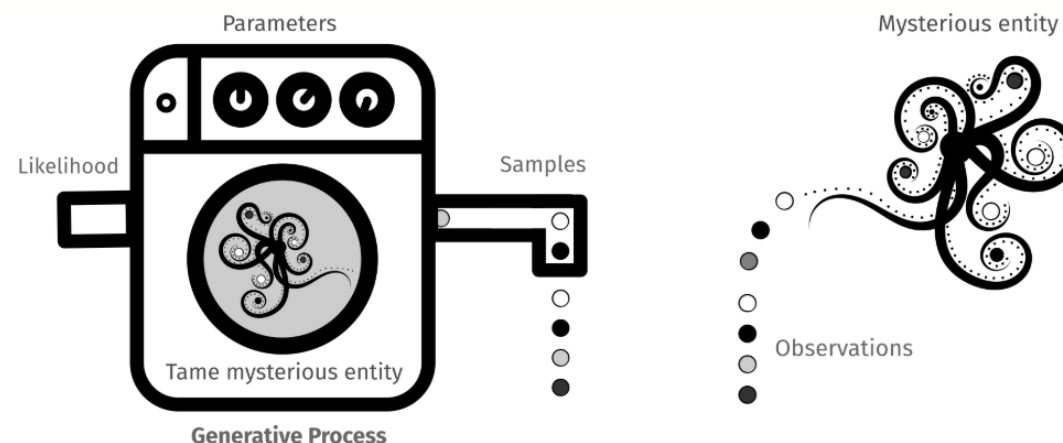
Optimisation solves all problems!

Even though we don't have a fixed, closed form function to estimate the parameters, with a likelihood function we can apply optimisation to work out a parameter setting under which the data we *actually* observed was most likely.

This corresponds to twiddling the knobs on our "mysterious entity" machine, until we find one that outputs the largest likelihood values when we feed in samples to it.

- If the likelihood depends on some parameters of a distribution θ , then we write:

$$\mathcal{L}(\theta|x)$$

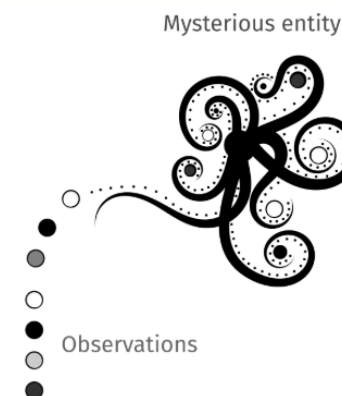
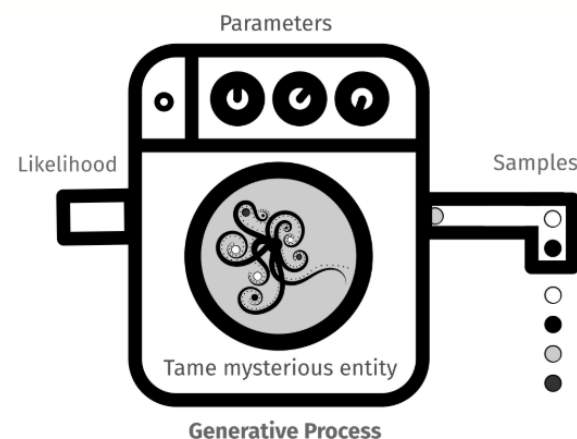


Optimisation solves all problems!

Then, we could define an **objective function**; to maximise the log-likelihood, or equivalently to minimise the negative log-likelihood.

$$\theta^* = \arg \min_{\theta} L(\theta) \quad L(\theta) = -\log \mathcal{L}(\theta|x_1, \dots, x_n) = -\sum_i \log f_X(x_i; \theta),$$

assuming our $f_X(x_i)$ can be written as $f(x_i; \theta)$ to represent the PDF of f with some specific choice of parameters given by θ .



Maximum Likelihood estimation

$$\theta^* = \arg \min_{\theta} L(\theta) \quad L(\theta) = -\log \mathcal{L}(\theta|x_1, \dots, x_n) = -\sum \log f_X(x_i; \theta),$$

This is *very* similar to the approximation objective function we saw before, $\|f(x; \theta) - y\|$, but

- we have $y=0$, and
- we only have a scalar output from f so the norm is unnecessary.

We already know how to solve this kind of problem; *just optimise!*

This is called **maximum likelihood estimation** and is a general technique for determining parameters of a distribution which we don't know given some observations.

It will find the **best** setting of parameters that would explain how the observations came to be.

Maximum Likelihood estimation

$$\theta^* = \arg \min_{\theta} L(\theta) \quad L(\theta) = -\log \mathcal{L}(\theta|x_1, \dots, x_n) = -\sum \log f_X(x_i; \theta),$$

- If we're lucky, this will be differentiable and we can use gradient descent (or stochastic gradient descent
 - note that the objective function is a sum of simple sub-objective functions).
- If we're not, we can fall back on less efficient optimisers.

We don't need special estimators in this case, as long as we can evaluate the PDF $f(x; \theta)$ for any setting of parameters θ .

This works for a much wider class of probability distributions.

Fitting a normal with MLE

- We can for example look at the problem of estimating the mean and variance of a normal distribution from a set of (assumed to be independent) samples *without* using estimators; for example our app ratings.
- To do this, we need to be able to compute the likelihood for any given sample, and take the product (or rather sum of log likelihoods) for all of those samples.
- This gives us our objective function. If we flip the sign, so that we minimise the negative log-likelihood, we will then search for the parameter vector that makes the data most likely.
- For a univariate normal distribution, the parameters are just μ and σ , so $\theta = [\mu, \sigma]$.
- In this case, of course, we *do* have estimators; but the procedure works just as well when we only have a likelihood function.



Fitting a normal with MLE

code

```
def loglik_norm(data, theta):  
    # compute log-likelihood of observations  
    # how likely is this data *under the assumption*  
    # that these parameters are set  
    return -np.sum(scipy.stats.norm(theta[0],  
                                     np.abs(theta[1])).logpdf(data))  
  
thetas = []  
for app,name in zip(apps, "ABC"):  
    print("MLE estimate for app {app}".format(app=name))  
  
    # compute the log-likelihood for this specific dataset  
    loglik = lambda theta, data=app: loglik_norm(data, theta)  
    # MLE optimise  
    theta = scipy.optimize.minimize(loglik,  
                                    [1.0, 1.0],  
                                    tol=1e-5).x  
  
    thetas.append(theta)  
    print("Sample statistics for app {app}".format(app=name))  
    print(theta)  
    # compare with using the direct estimates of these parameters  
    print(np.mean(app), np.std(app))
```

output

```
MLE estimate for app A  
Sample statistics for app A  
[3.15999998 1.06508213]  
3.16 1.0650821564555477
```

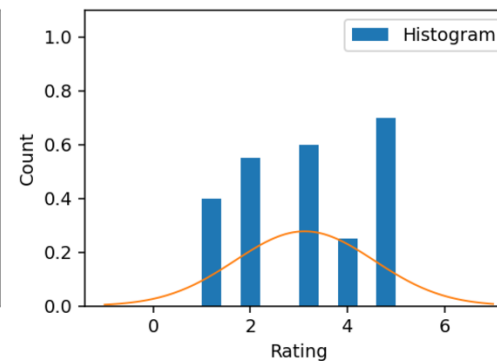
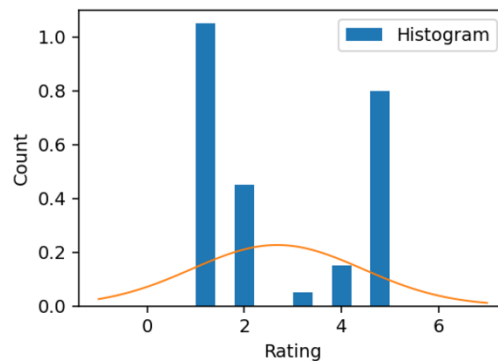
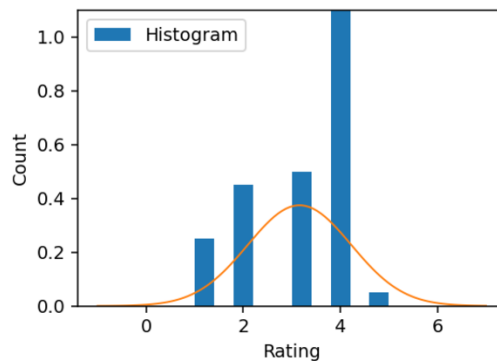
```
MLE estimate for app B  
Sample statistics for app B  
[2.67999983 1.75999998]  
2.68 1.76
```

```
MLE estimate for app C  
Sample statistics for app C  
[3.12000007 1.43721952]  
3.12 1.4372195378577344
```

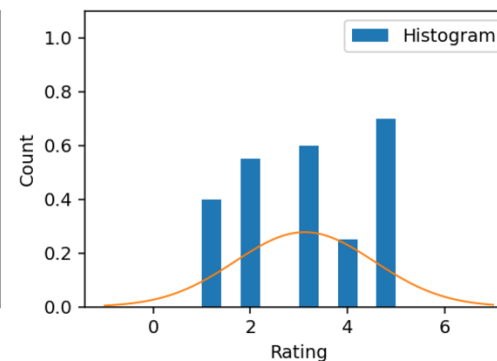
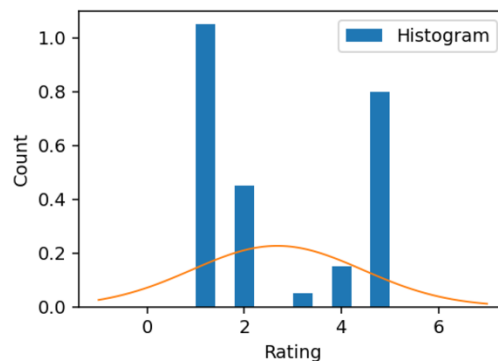
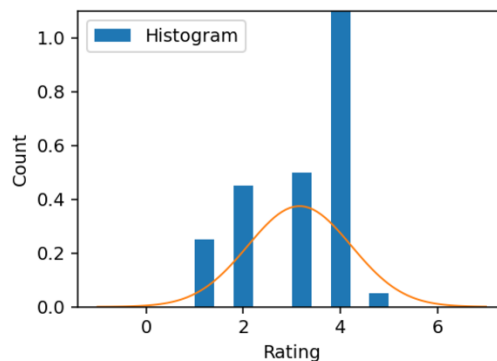


Fitting a normal with MLE

```
[21]: # use the sample mean, sample standard deviation
plot_fits(apps, [[np.mean(app)] for app in apps], [[np.std(app)] for app in apps])
plt.show()
```



```
[22]: # Use the MLE fit, which should look exactly the same (within some tiny tolerance)
plot_fits(apps, [[theta[0]] for theta in thetas], [[theta[1]] for theta in thetas])
plt.show()
```





A mixture model

But what if our model was more complicated than just a normal distribution?

We could imagine that we model in some other way, perhaps that might be able to capture the fact that app B seems to have two "humps" on either side.

One very simple model is a **mixture of Gaussians**, where we just say that we expect the PDF of the distribution we are trying to fit is a weighted combination (convex sum) of N different normal distributions ("components") $N_i(\mu_i, \sigma_i)$, each with its own μ_i, σ_i , and with a weighting factor λ_i that says how important this "component" is, where $\sum_i \lambda_i = 1$.

This lets us represent "humpy" distributions.



A mixture model

This model lets us imagine that ratings might belong to one "cluster" or another.

placement and size of each cluster is given by the μ_i and σ_i for that component and λ_i gives an idea of how likely data is to fall into that cluster.

We can easily plot the PDF of this function; it's just:

$$f_X(x) = \sum_i \lambda_i n_X(x; \mu_i, \sigma_i),$$

where

$$n_X(x; \mu, \sigma) = \frac{1}{Z} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

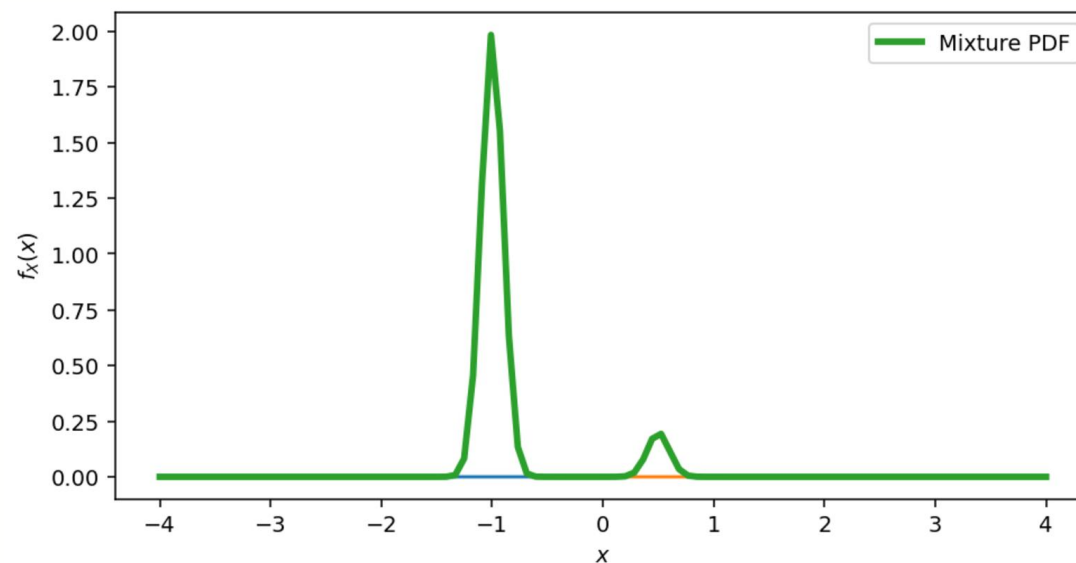
is the standard normal PDF function



Mixture model: Two components

```
[23]: def pdf_mixture(mus, sigmas, lambdas):  
    fig = plt.figure()  
    ax = fig.add_subplot(1,1,1)  
    xs = np.linspace(-4,4,100)  
    pdf = np.zeros_like(xs)  
  
    # compute the PDF of each component, weighted by  
    # the corresponding lambda  
    for mu, sigma, l in zip(mus, sigmas, lambdas):  
        # sum of pdfs  
        sub_pdf = scipy.stats.norm(mu, sigma).pdf(xs) * l  
        pdf += sub_pdf  
        ax.plot(xs, sub_pdf)  
  
    # plot the sum PDF  
    ax.plot(xs, pdf, lw=3, label="Mixture PDF")  
    ax.legend()  
    ax.set_xlabel("$x$")  
    ax.set_ylabel("$f_X(x)$")
```

```
[24]: # N = 2  
pdf_mixture([0.5, -1], [0.1, 0.1], [0.05, 0.5])  
plt.show()
```

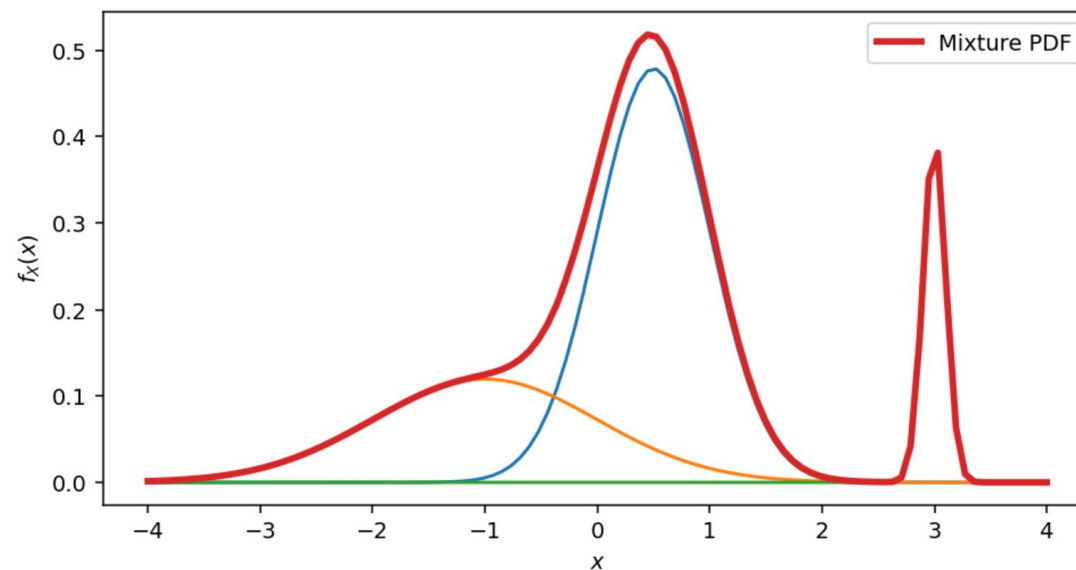




Mixture model: Three components

```
[23]: def pdf_mixture(mus, sigmas, lambdas):  
    fig = plt.figure()  
    ax = fig.add_subplot(1,1,1)  
    xs = np.linspace(-4,4,100)  
    pdf = np.zeros_like(xs)  
  
    # compute the PDF of each component, weighted by  
    # the corresponding lambda  
    for mu, sigma, l in zip(mus, sigmas, lambdas):  
        # sum of pdfs  
        sub_pdf = scipy.stats.norm(mu, sigma).pdf(xs) * l  
        pdf += sub_pdf  
        ax.plot(xs, sub_pdf)  
  
    # plot the sum PDF  
    ax.plot(xs, pdf, lw=3, label="Mixture PDF")  
    ax.legend()  
    ax.set_xlabel("$x$")  
    ax.set_ylabel("$f_X(x)$")
```

```
[25]: # N = 3  
pdf_mixture([0.5, -1, 3],  
            [0.5, 1.0, 0.1],  
            [0.6, 0.3, 0.1])  
plt.show()
```





Fitting mixtures

This is a much more plausible model of our app ratings, and might be a much better model.

But how do we fit it?

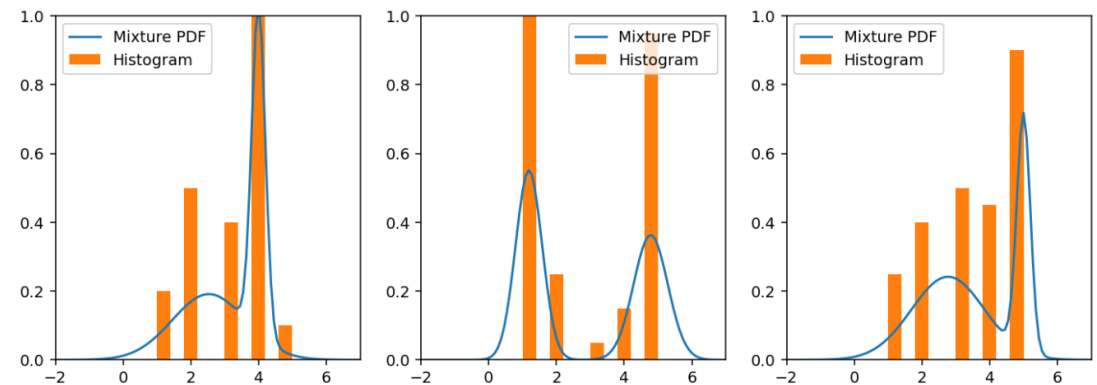
Even if we fix N in advance, we definitely don't have any direct estimators that can estimate the mean and standard deviation (and weighting) of a sum of normal PDFs.

This simply isn't something we know how to do.

But the (log) likelihood is trivial to write in code:

- For each observation x , we just compute the sum of the weighted PDFs for each component, and the result is likelihood for that observation.
- This is a function of the data $L(\theta|x)$, and our parameter vector is $\theta = [\mu_1, \sigma_1, \lambda_1, \mu_2, \sigma_2, \lambda_2, \dots]$.

```
[26]: def pdf_mixture(xs, mus, sigmas, lambdas):  
        return sum([scipy.stats.norm(mu, sigma).  
                    pdf(xs) * l  
                    for mu, sigma, l in  
                    zip(mus, sigmas, lambdas)])  
  
# Log Likelihood of the mixture PDF  
def llik_pdf_mixture(xs, mus, sigmas, lambdas):  
    return np.sum(np.log(pdf_mixture(xs, mus, sigmas, lambdas)))
```





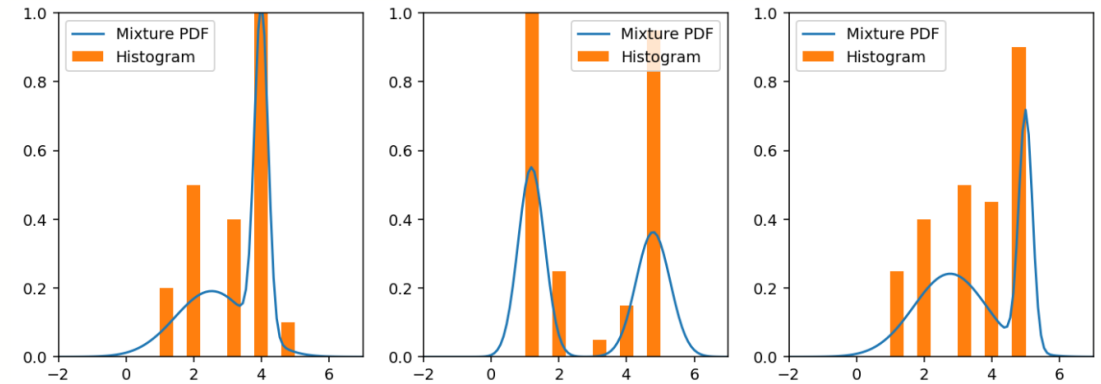
Fitting mixtures

This means we can also fit it with maximum likelihood.

We do not require an explicit estimator for the parameters; we can just optimise.

In this case, we fix N to 2, and then find the parameters that best fit the data.

This is a point in the vector space $\mathbb{R}^6$; we need three parameters $\mu_i, \sigma_i, \lambda_i$ for each component i .





C. Bayesian Inference

Bayesian Inference

Form guesses over possible knob states

Update guesses based on likelihood

Result in **distributions** over knob settings

Bayes' Rule

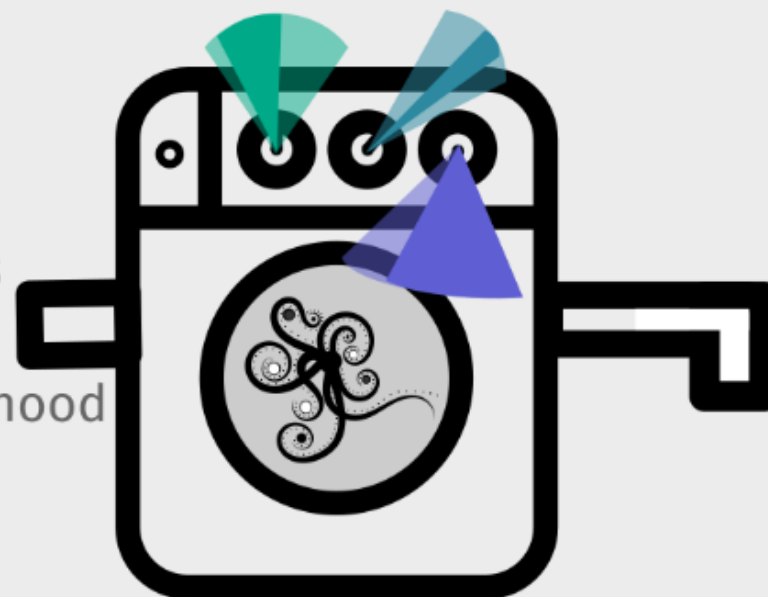
$$P(A|B) \propto P(B|A)P(A)$$

Observations



Likelihood

Update knob **beliefs**
Prior -> Posterior





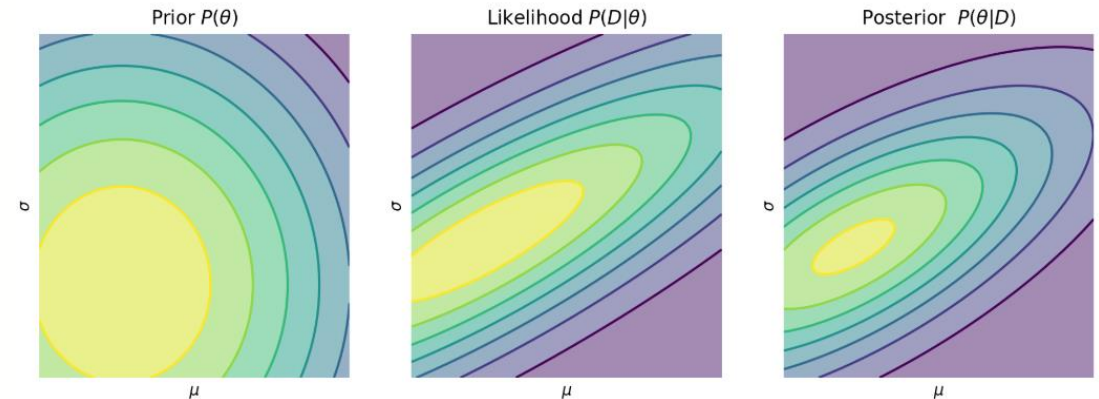
Bayesian Inference

We talk about inferring a **posterior** distribution over the parameters, given some **prior** belief and some **evidence**.

We assume that we have a **likelihood function** $P(D|\theta)$, and a prior over parameters $P(\theta)$ and we can then use Bayes Rule in the form:

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$$

which gives us a new distribution over θ given some observations.





Example

Remember, we are assuming that app ratings are generated by this function:

```
[30]: def app_samples(mu, sigma):  
       return np.random.normal(mu, sigma)
```

We want to *infer a distribution* over μ and σ (**NOT** a distribution over the observations!).

That is, we will treat the parameters themselves as random variables, with their own distributions, and use Bayesian reasoning (i.e. applying Bayes Rule) to infer a posterior distribution over the parameters given some prior, and some evidence observed.



Example

Parameters and samples

- We have a collection of observations $D=x_1, x_2, \dots$, which represent actual app ratings.
- We're not sure how much those ratings really tell us about the unseen population of potential users.
- We represent the distribution parameters as $\theta=[\mu, \sigma]$, and can talk about $P(\theta)$, a distribution over the parameter vectors.

Priors

- Assume we have some prior belief θ , $P(\theta)$: for example this might be a very simple assumption that our prior is that μ and σ are uniformly distributed
 - $\mu \sim U(1, 5)$
 - $\sigma \sim U(0, 10)$
- This gives us a form for $P(\theta)$.

Likelihood

We need to be able to define a **likelihood function**. This is a function of data given some parameter setting, and in this case it is the same as the likelihood function used for MLE: the likelihood of one sample is just the normal PDF evaluated at that point, and the likelihood of all samples is the product of these likelihoods.

```
[33]: def log_likelihood(samples, theta):  
      # for a *known* mu, theta  
      # compute the log PDF at each sample, and sum them  
      return np.sum(scipy.stats.norm(theta[0],  
                                     theta[1]).logpdf(samples))
```

Note carefully: in many cases we can only evaluate this likelihood function directly for a *specific* setting of θ ; but we have a **distribution** over θ to deal with when doing Bayesian inference.



Inference

How can we compute the posterior distribution $P(\vartheta|D)$?

We won't discuss how to find this in closed form (as a function) but rather how to draw samples from this posterior, given a prior and a likelihood and some observations.

There is a huge literature on how to solve this problem, which has a few nasty parts:

- $P(D|\theta)$ needs to be computed for a **distribution** over θ , not just some numbers.
- $P(D) = \int_{\theta} P(D|\theta)P(\theta)$ which is likely intractable.



Making it tractable: two simplifications

Samples will do

- We often can't compute $P(\theta | D)$ because we don't know how to do operations on products of functions. But it's often trivial for *specific, concrete* values of θ .
- This leads us to the idea of **drawing samples** from the posterior distribution $P(\theta | D)$, instead of trying to compute the distribution exactly.

Making it tractable: two simplifications

Samples will do

- We often can't compute $P(\theta | D)$ because we don't know how to do operations on products of functions. But it's often trivial for *specific, concrete* values of θ .
- This leads us to the idea of **drawing samples** from the posterior distribution $P(\theta | D)$, instead of trying to compute the distribution exactly.

Relative probability only

- We can make a simplifying assumption: we only care about the *relative* probability of different parameter settings with the data that we actually have, D .

That is we have $P(\theta | D) \propto P(D | \theta)P(\theta)$

and ignore the fact that this is the posterior scaled by some unknown constant $Z = \frac{1}{P(D)}$

- This only makes sense because we are only considering one model with one set of data in this example.

Markov Chain Monte Carlo

We can implement a procedure to sample from the (relative) posterior distribution via a very simple modification of the *simulated annealing* algorithm.

This defines a random walk through the space of parameter settings, proposing small random tweaks to the parameter settings, and accepting "jumps" if they make the estimate more likely, or with a probability proportional to the change in $P(D|\theta)P(\theta)$ if not.

The advantage of this approach is that we can work with *definite samples* from θ and we don't have to do any tricky integrals.

This approach is called **Markov Chain Monte Carlo**

All we require is a way of evaluating $P(\theta)$ (prior) and $P(D|\theta)$ (likelihood) for any specific θ .

MCMC in practice: sampling issues

We will use Markov Chain Monte Carlo to solve the Bayesian inference problem. The **great thing** about MCMC approaches is that you can basically write down your model and then run inference directly. There is no need to derive complex approximations, or to restrict ourselves to limited models for which we can compute answers analytically. Essentially, no maths by hand; everything is done algorithmically.

MCMC allows us to draw samples from any distribution $P(X=x)$ *that we can't sample from directly*. In particular, we will be able to sample from the posterior distribution over parameters.

The **bad thing** about MCMC approaches is that, even though it will do the "right thing" *asymptotically*, the choice of sampling strategy has a very large influence for the kind of sample runs that are practical to execute. Bayesian inference should depend only on the priors and the evidence observed; but MCMC approaches also depend on the sampling strategy used to approximate the posterior.

What distribution are we sampling from?

In the case of Bayesian inference

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)} = \frac{P(D|\theta)P(\theta)}{\int_{\theta} P(D|\theta)P(\theta)}.$$

- $P(\theta|D)$ is the posterior, the distribution over the parameters θ given the data (observations) D , using:
- the likelihood $P(D|\theta)$,
- prior $P(\theta)$ and
- evidence $P(D)$.

In other words, *what is the distribution over the parameters given the observations and the prior?*

If we assume, as above, that we don't care about $P(D)$, because we are only comparing different possible values of θ then we can draw samples from a distribution proportional to $P(D|\theta)P(\theta)$.

Metropolis-Hastings

Metropolis-Hastings (or just plain Metropolis) is a wonderfully elegant and relatively effective way of doing this MCMC algorithm, and is able to work in high-dimensional spaces (i.e. when θ is complicated).

- Metropolis sampling uses a simple auxiliary distribution called the **proposal distribution** $Q(\theta' | \theta)$ to help draw samples from an intractable posterior distribution $P(\theta | D)$.
- This is analogous to what we called the **neighbourhood function** in the optimisation section.
- Metropolis-Hastings uses this to **wander around** in the distribution space, accepting jumps to new positions using $Q(\theta' | \theta)$ to randomly sample the space of $P(\theta | D)$. This random walk (a **Markov chain**, because we make a random jump conditioned only on where we currently are) is the "Markov Chain" bit of "Markov Chain Monte Carlo".



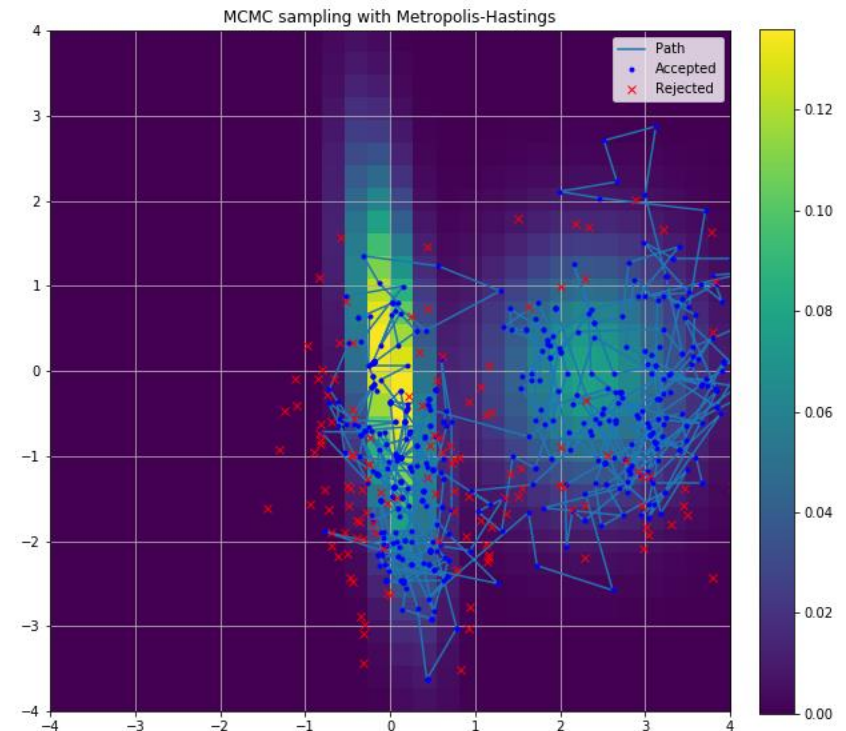
Metropolis-Hastings

This is just like the simulated annealing algorithm, except now there is a function $f_x(\theta)$ which makes some steps more likely than others instead of a likelihood function.

We just take our current position θ , and propose a new position θ' , that is a random sample drawn from $Q(\theta'|\theta)$.

Often this is something simple like a normal distribution with mean x and some preset σ :

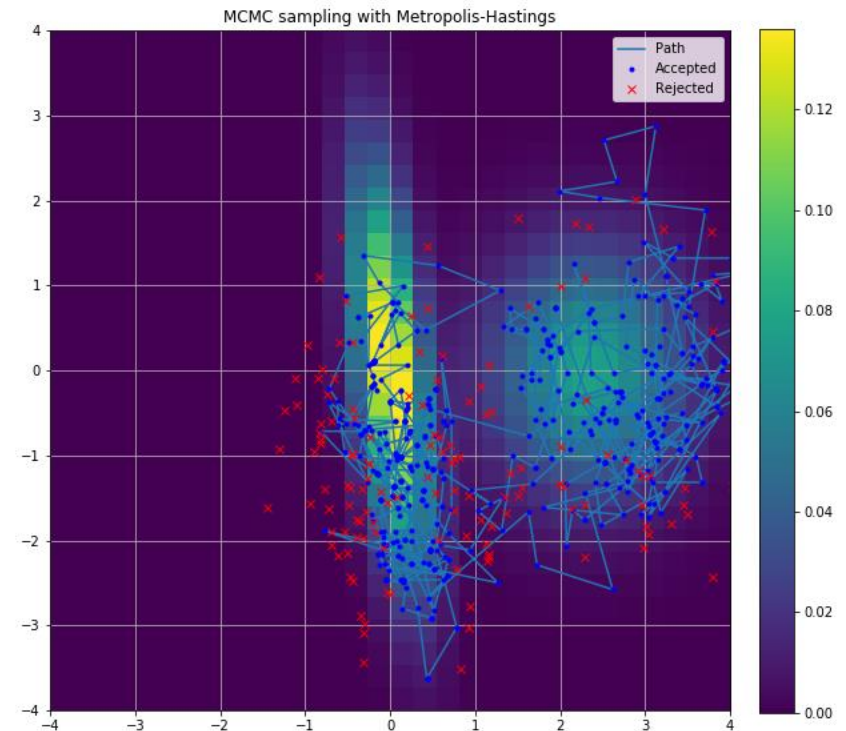
$$Q(\theta'|\theta) = \mathcal{N}(\theta, \sigma')$$





Trace

- The history of accepted samples of an MCMC process is called the **trace**.
- We can estimate model parameters by looking at the histogram of the **trace**, for example.
- The trace is the sequence of samples $[x(1), x(2), x(3), \dots, x(n)]$, (approximately) drawn from the **posterior** distribution $P(\theta|D)$ via MCMC.





Applying MCMC

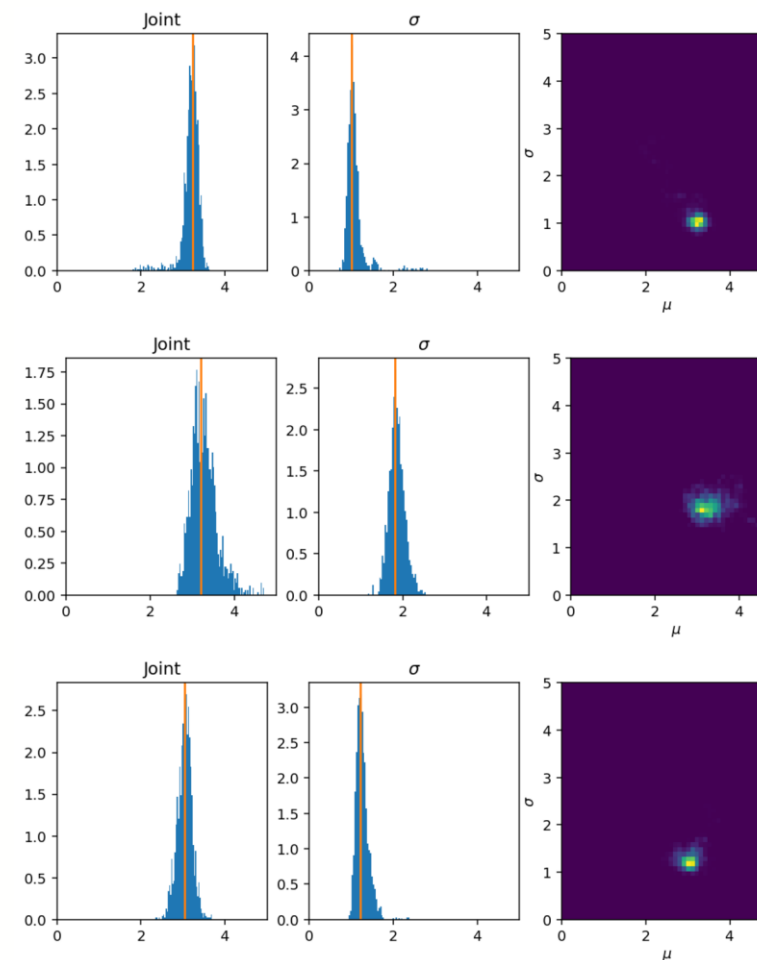
```
[34]: from metropolis import log_metropolis

traces = []
for app in apps:
    init_guess = sample_prior()
    log_fx = lambda theta, data=app: log_prior(theta) + log_likelihood(data, theta)
    q = lambda x: x + np.random.normal(0, 0.08, size=2) # a small random step
    # apply our algorithm
    accepted = log_metropolis(log_fx, q, init_guess, 2000)
    traces.append(accepted)
```

Acceptance ratio: 0.717

Acceptance ratio: 0.827

Acceptance ratio: 0.775





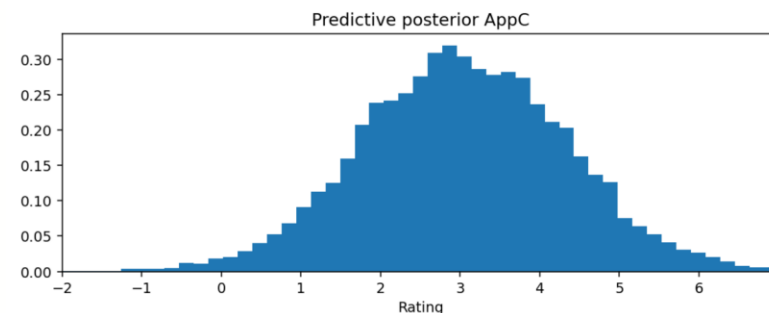
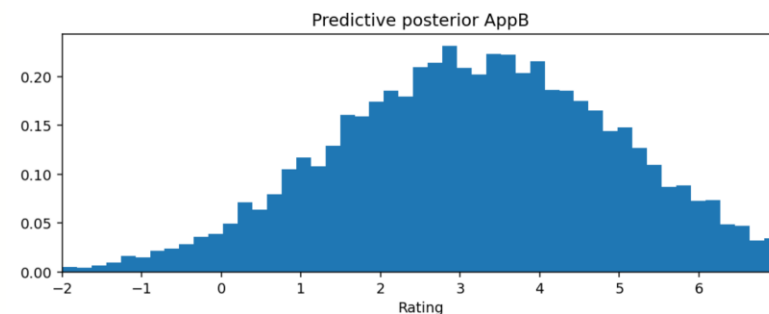
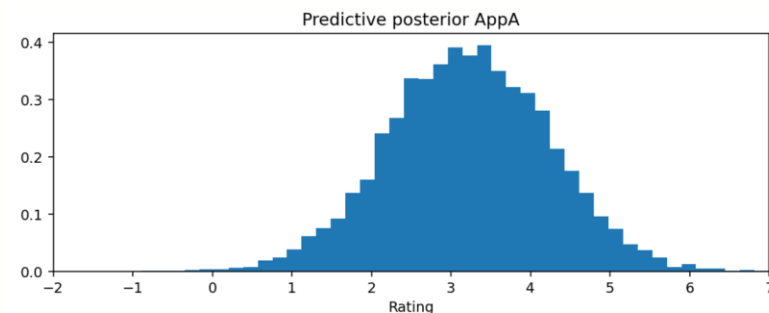
Predictive posterior: sampling from the model

What we have plotted is samples from the **posterior distribution of the model parameters**; i.e. the values we expect the model parameters to take on given the data we observed and our prior.

The **predictive posterior** is the **distribution over observations** we would expect to see; predictions of future samples. This means drawing samples from the model, while integrating over parameters from the posterior.

We can do this with a two-step, nested process:

- for n repetitions
 - draw samples from our posterior distribution over parameters to give us a concrete distribution
- for m repetitions
 - draw samples from this concrete distribution





University
of Glasgow

Thank you

**Contact:
Nicolas.Pugeault@Glasgow.ac.uk**